

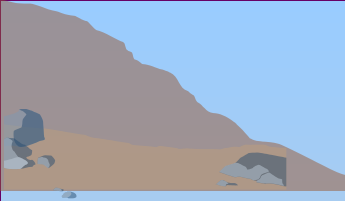
Lecture 8

Topics

- Depth First Search (DFS)
- Breadth First Search (BFS)

Reference: Introduction to Algorithm by Cormen

Chapter 23: Elementary Graph Algorithm

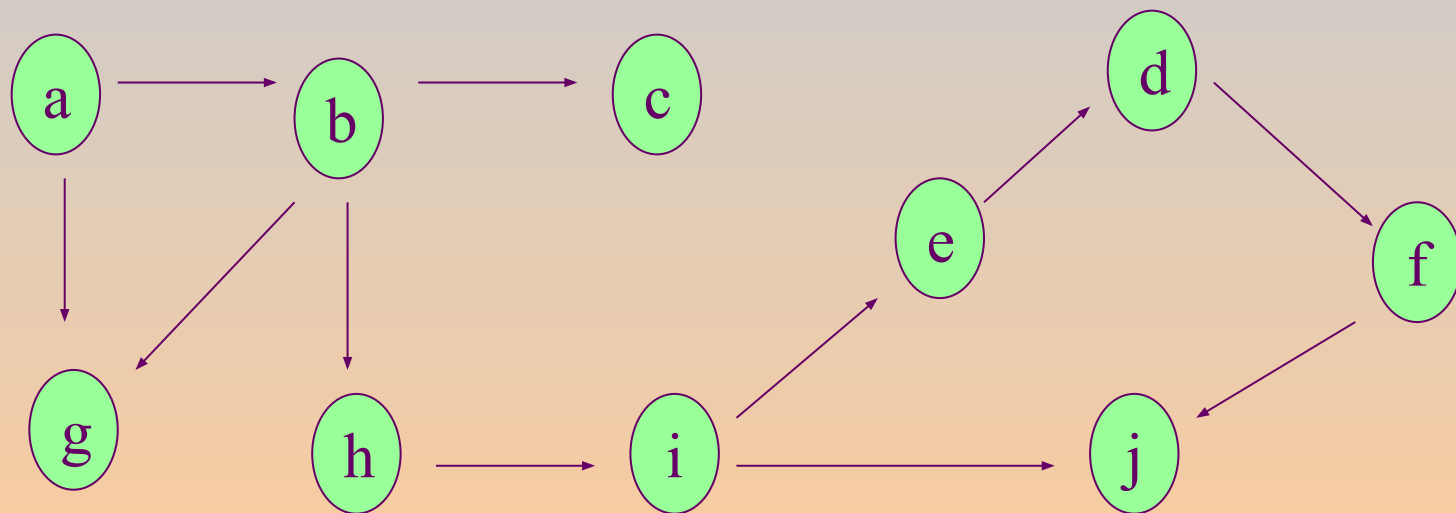


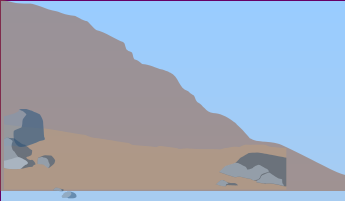
Graph Search (traversal)

- How do we search a graph?
 - At a particular vertices, where shall we go next?
- Two common framework:
 - the depth-first search (DFS)
 - the breadth-first search (BFS) and
- In DFS, go as far as possible along a single path until reach a dead end (a vertex with no edge out or no neighbor unexplored) then backtrack
- In BFS, one explore a graph level by level away (explore all neighbors first and then move on)

Depth-First Search (DFS)

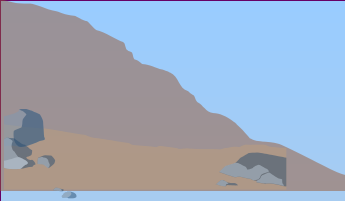
- The basic idea behind this algorithm is that it traverses the graph using recursion
 - Go as far as possible until you reach a deadend
 - Backtrack to the previous path and try the next branch
 - The graph below, started at node a, would be visited in the following order: a, b, c, g, h, i, e, d, f, j





DFS: Color Scheme

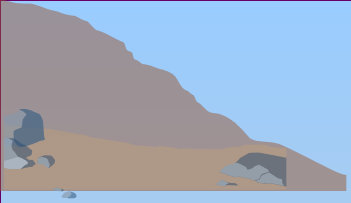
- Vertices initially colored white
- Then colored gray when discovered
- Then black when finished



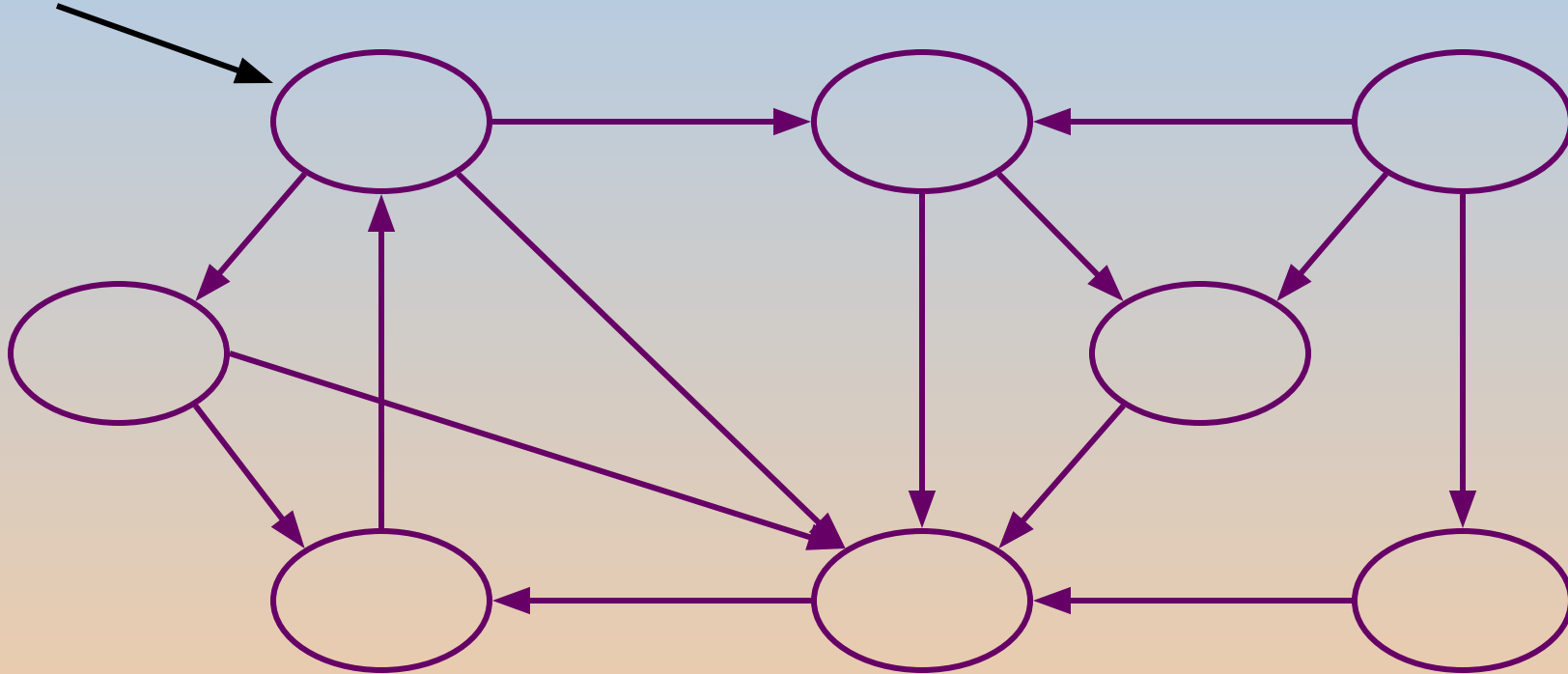
DFS: Time Stamps

- Discover time $d[u]$: when u is first discovered
- Finish time $f[u]$: when backtrack from u
- $d[u] < f[u]$

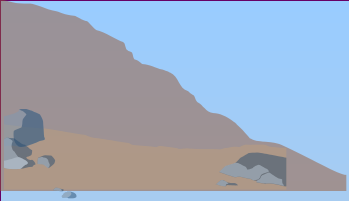
DFS Example



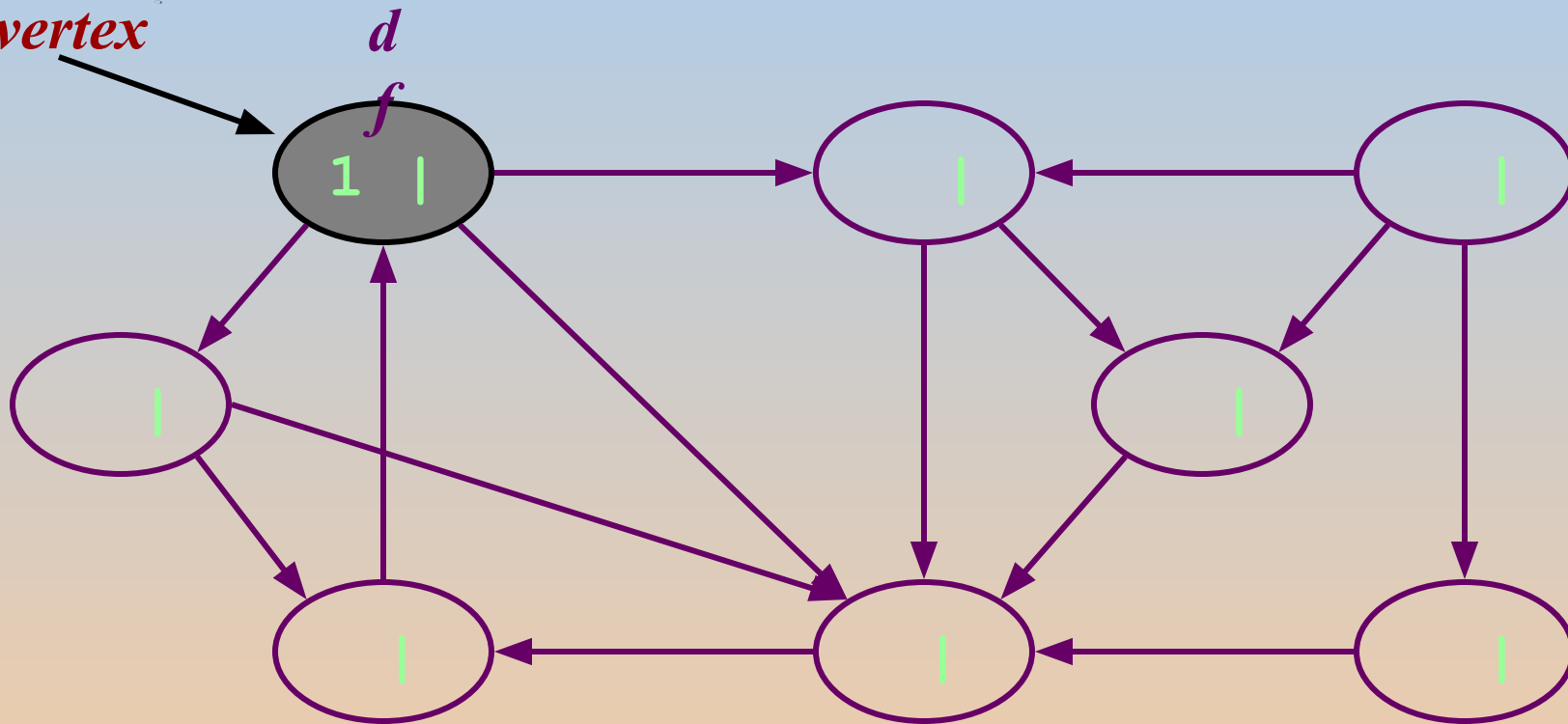
*source
vertex*



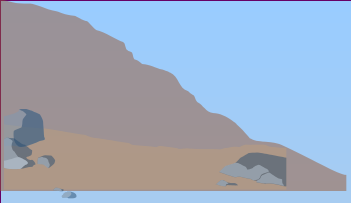
DFS Example



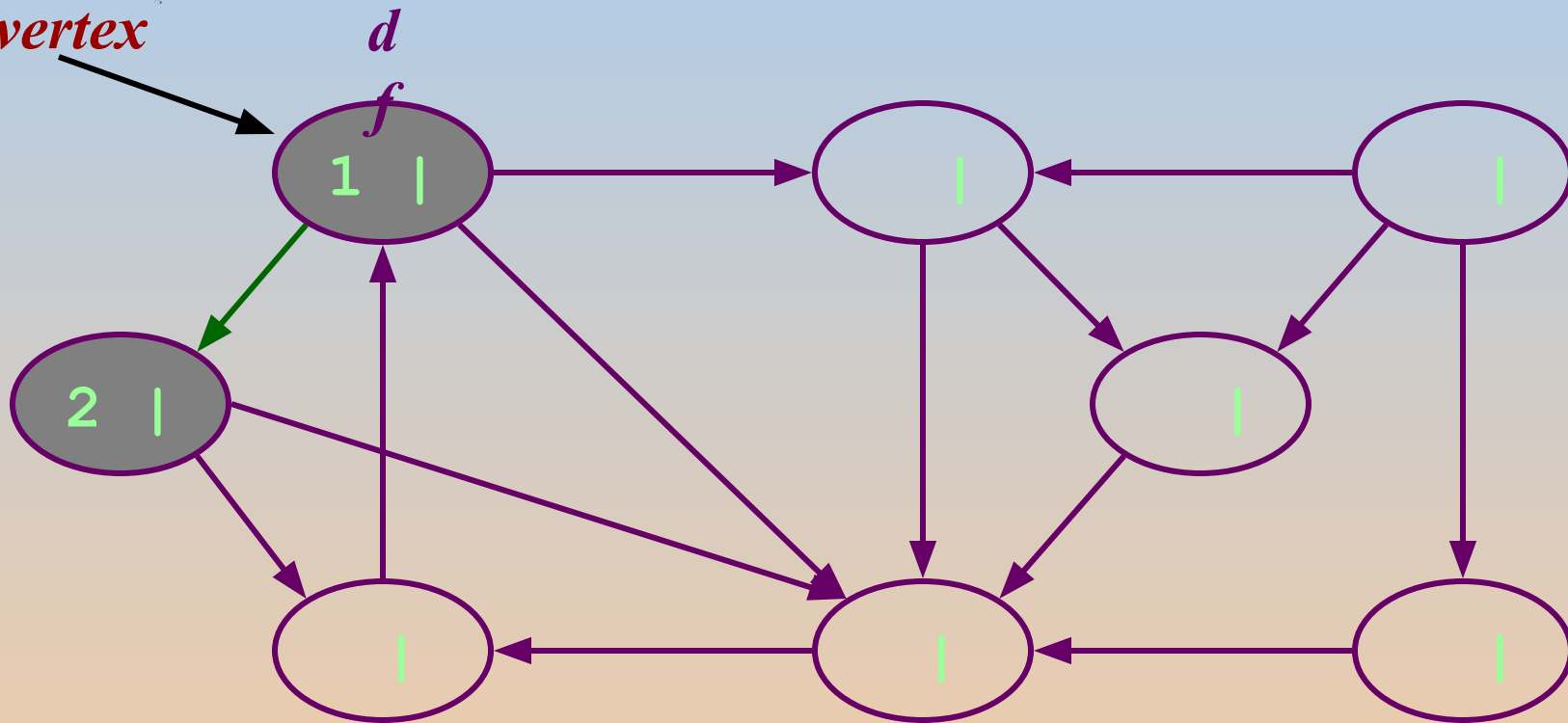
*source
vertex*



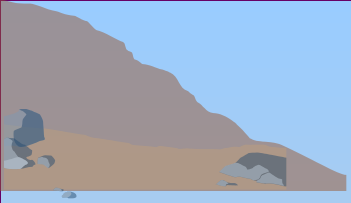
DFS Example



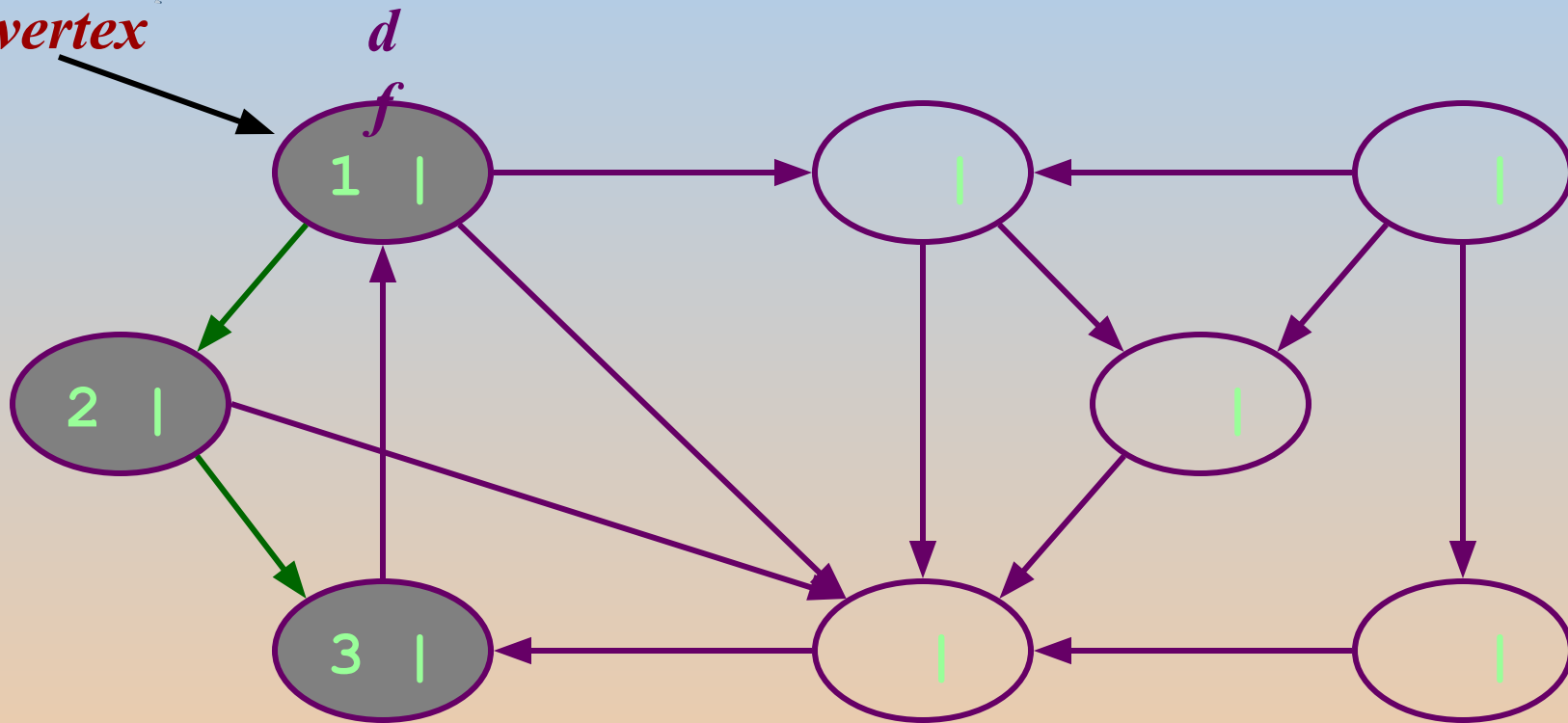
*source
vertex*



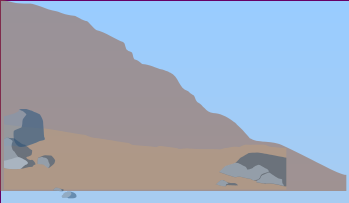
DFS Example



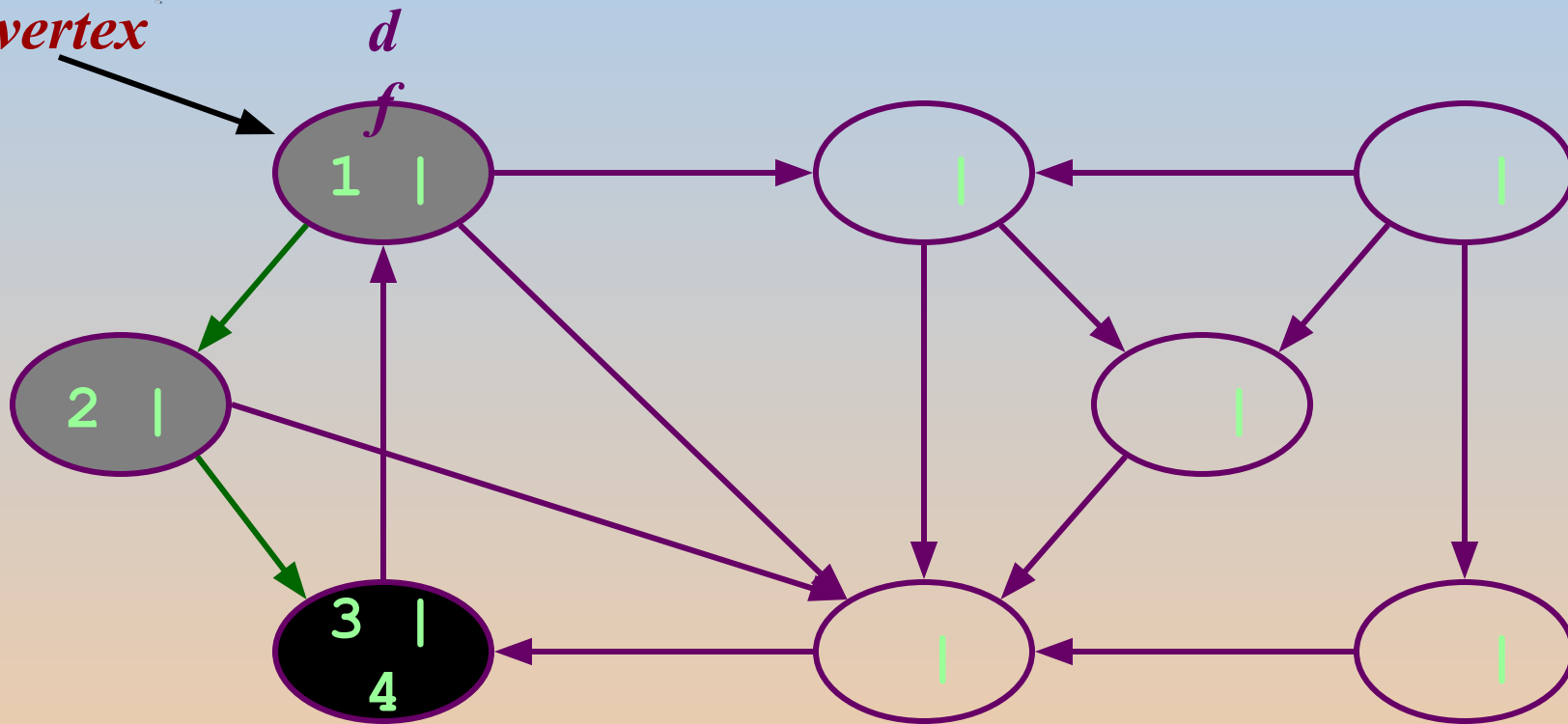
*source
vertex*



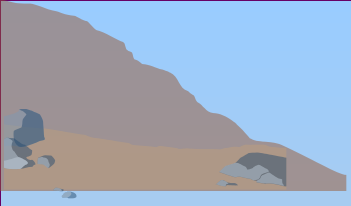
DFS Example



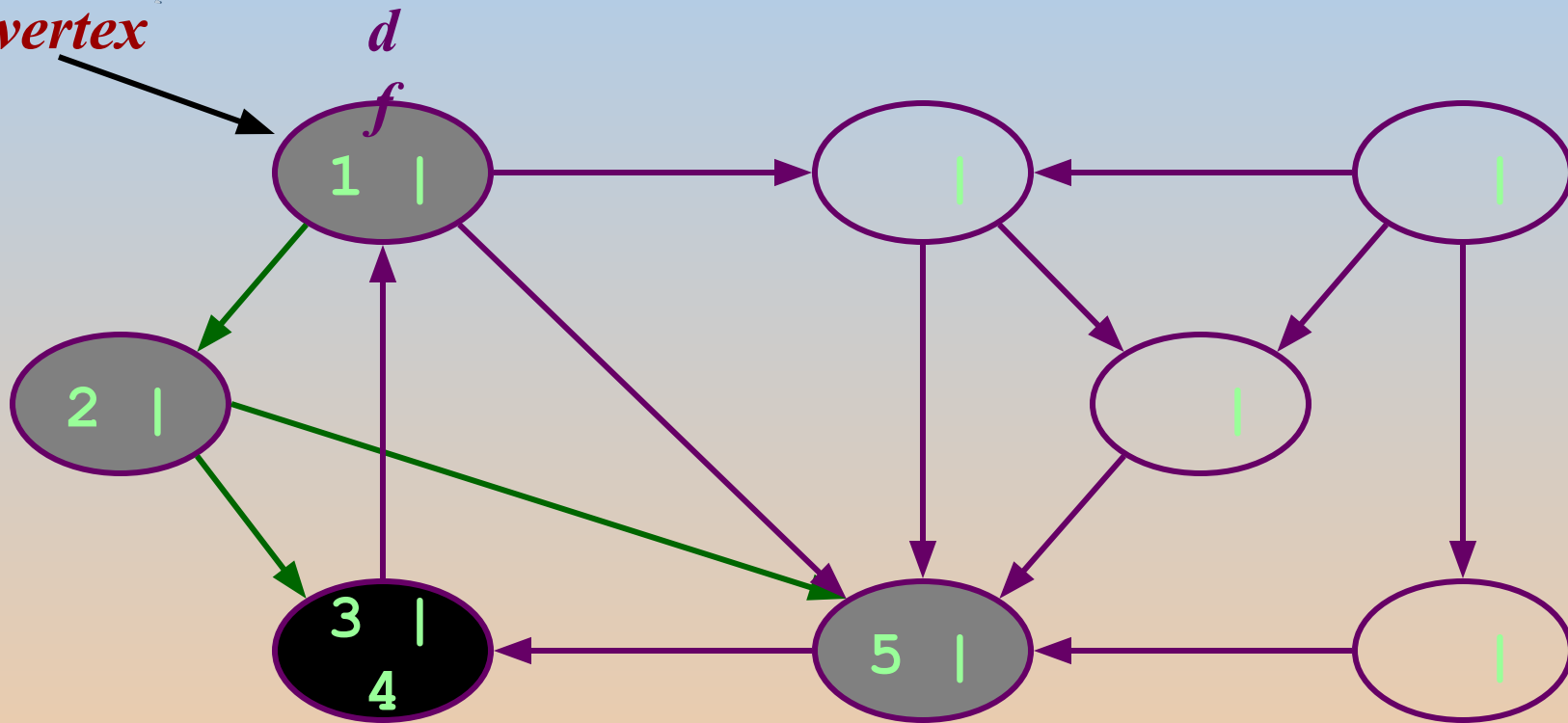
*source
vertex*



DFS Example

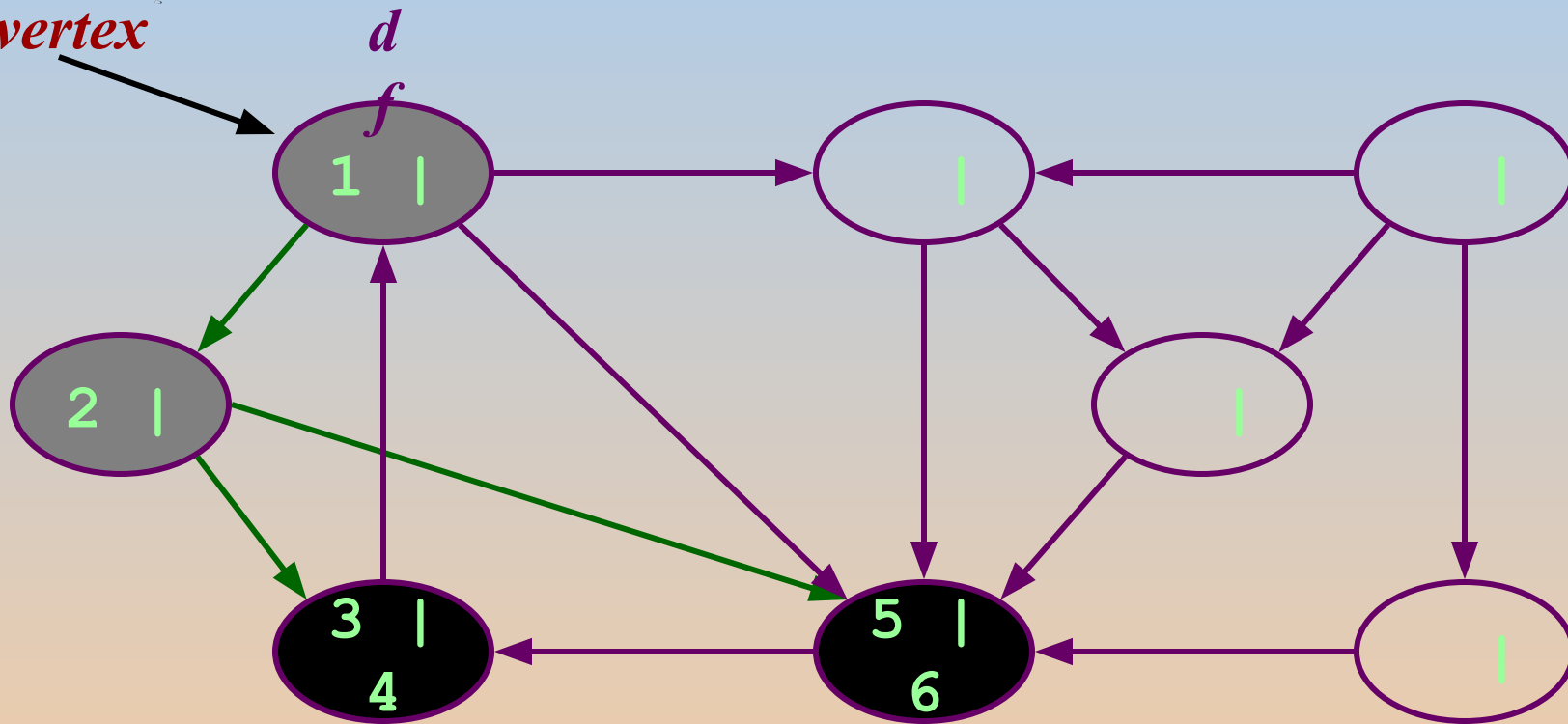


*source
vertex*



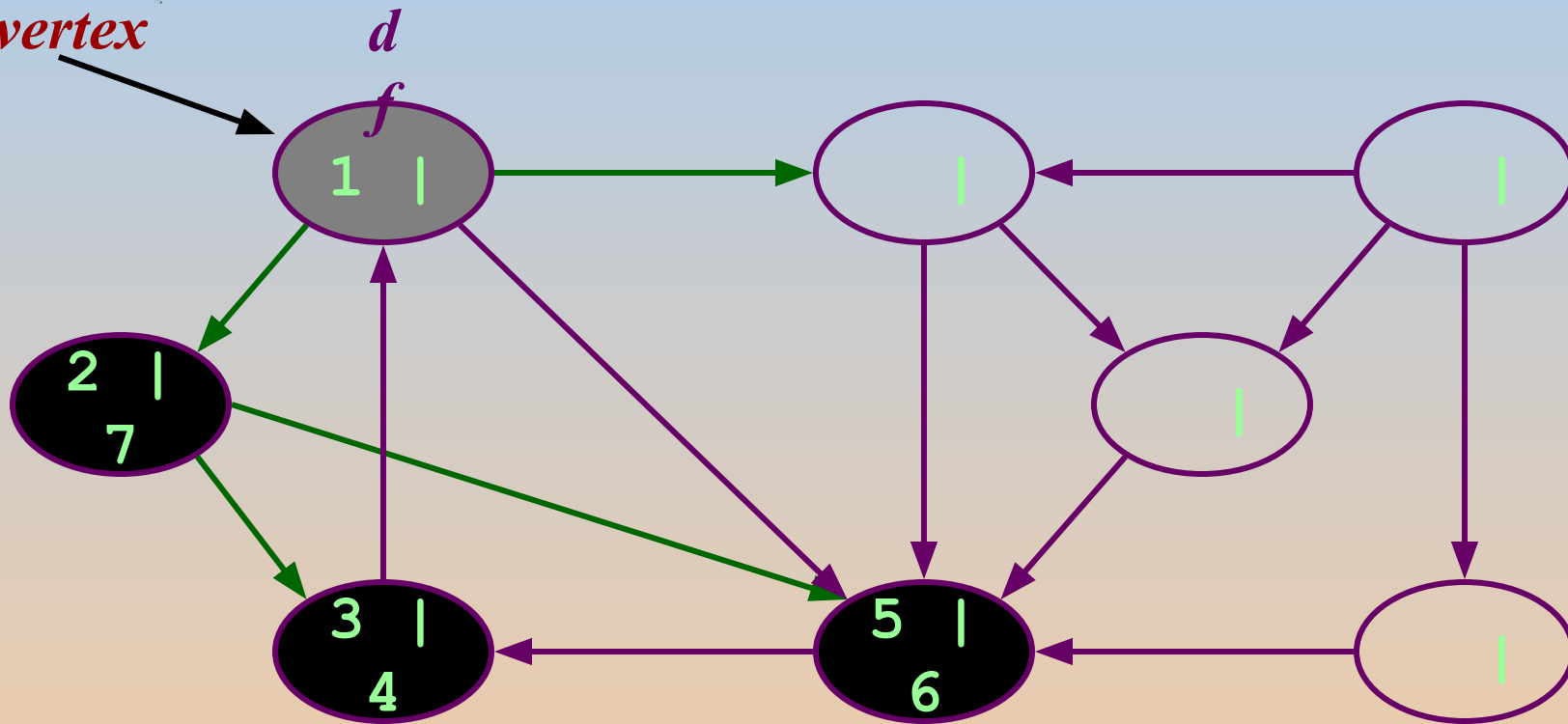
DFS Example

source
vertex



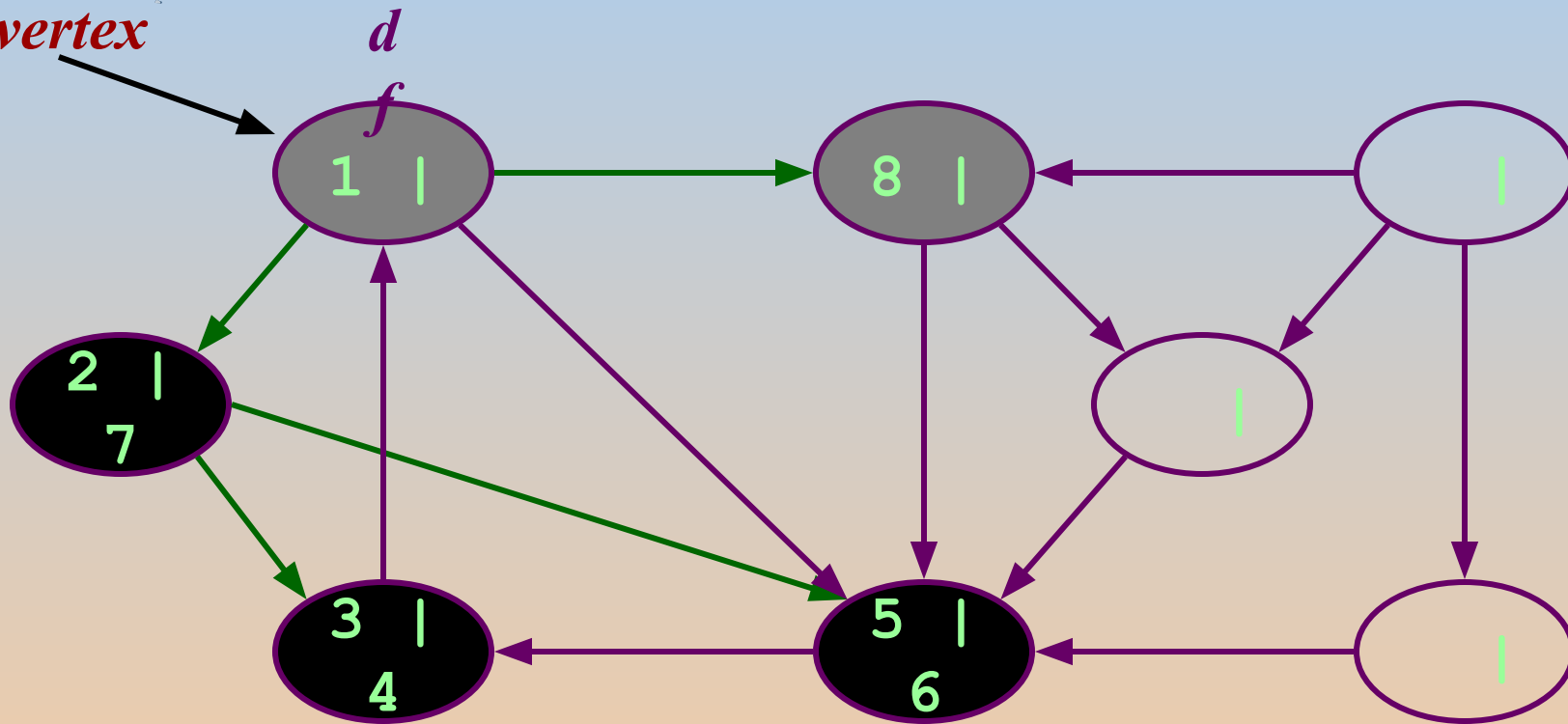
DFS Example

*source
vertex*



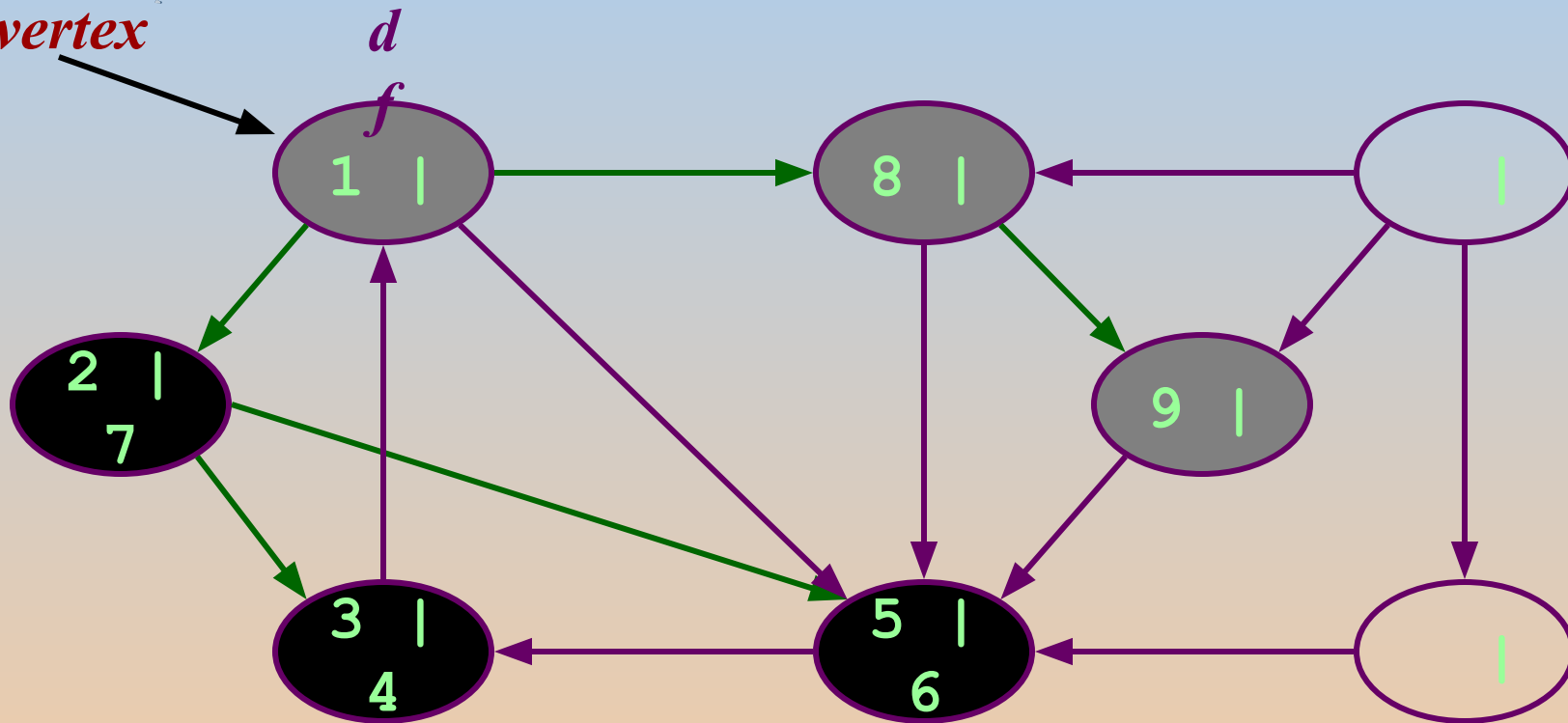
DFS Example

*source
vertex*



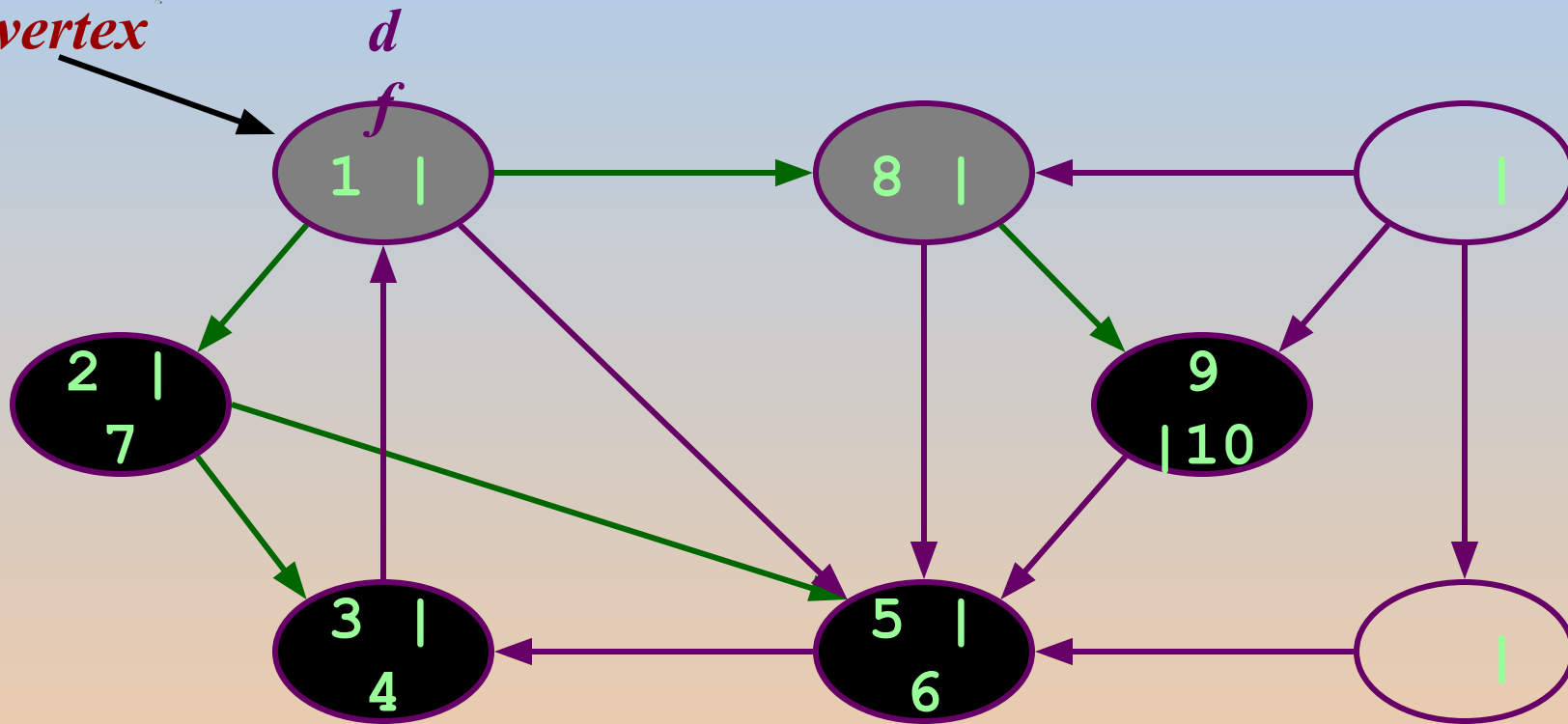
DFS Example

source
vertex

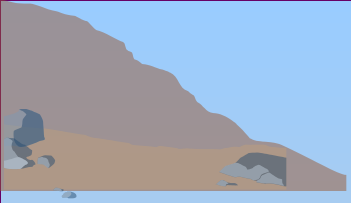


DFS Example

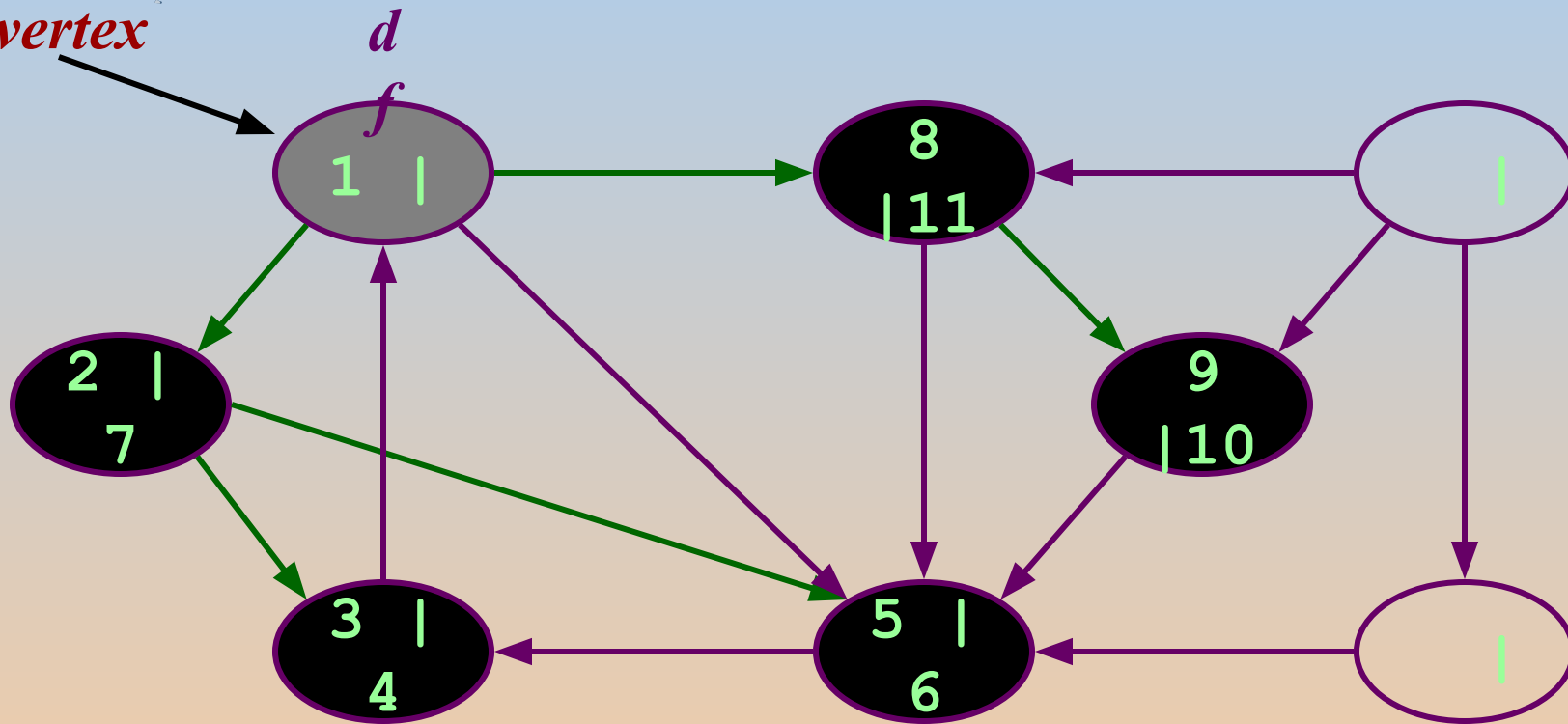
*source
vertex*



DFS Example

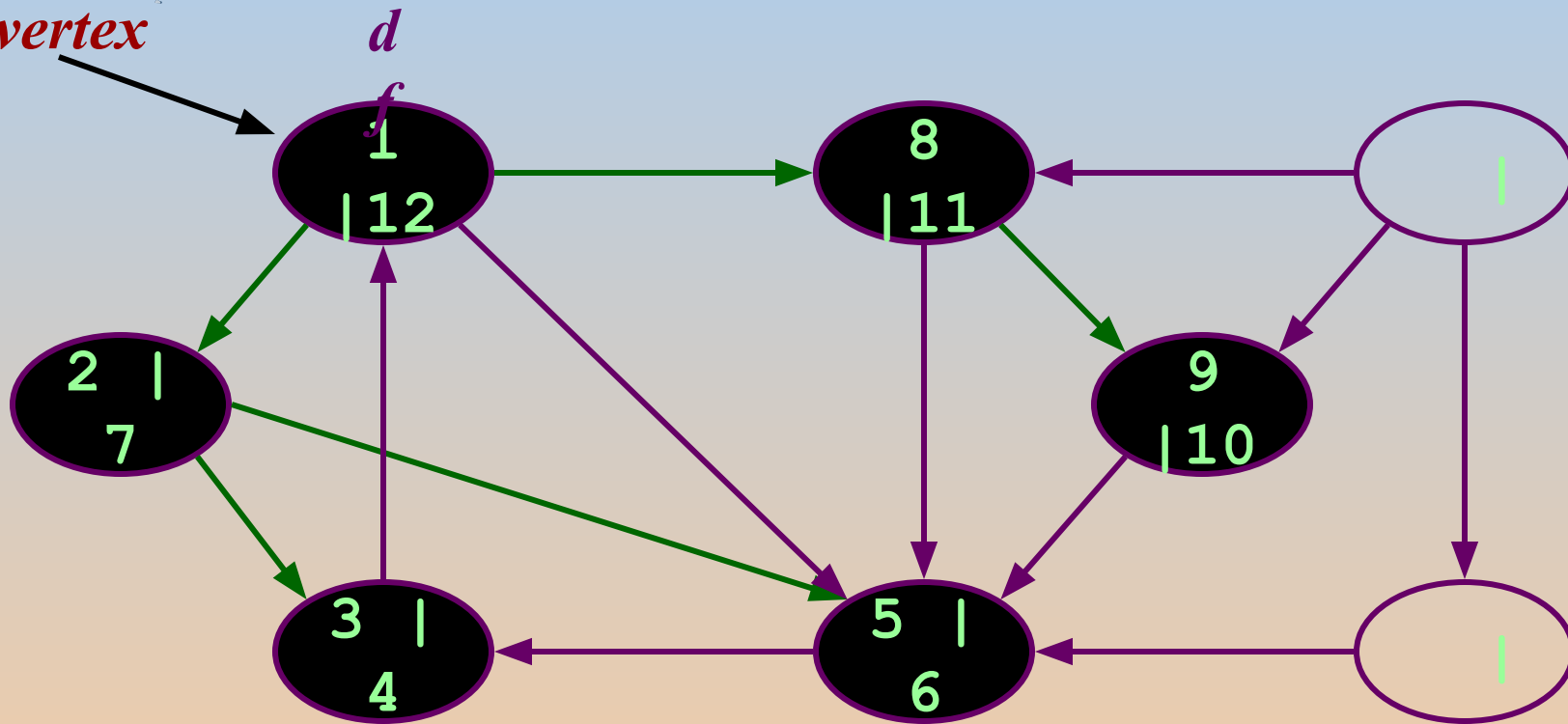


*source
vertex*

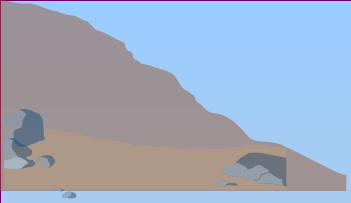


DFS Example

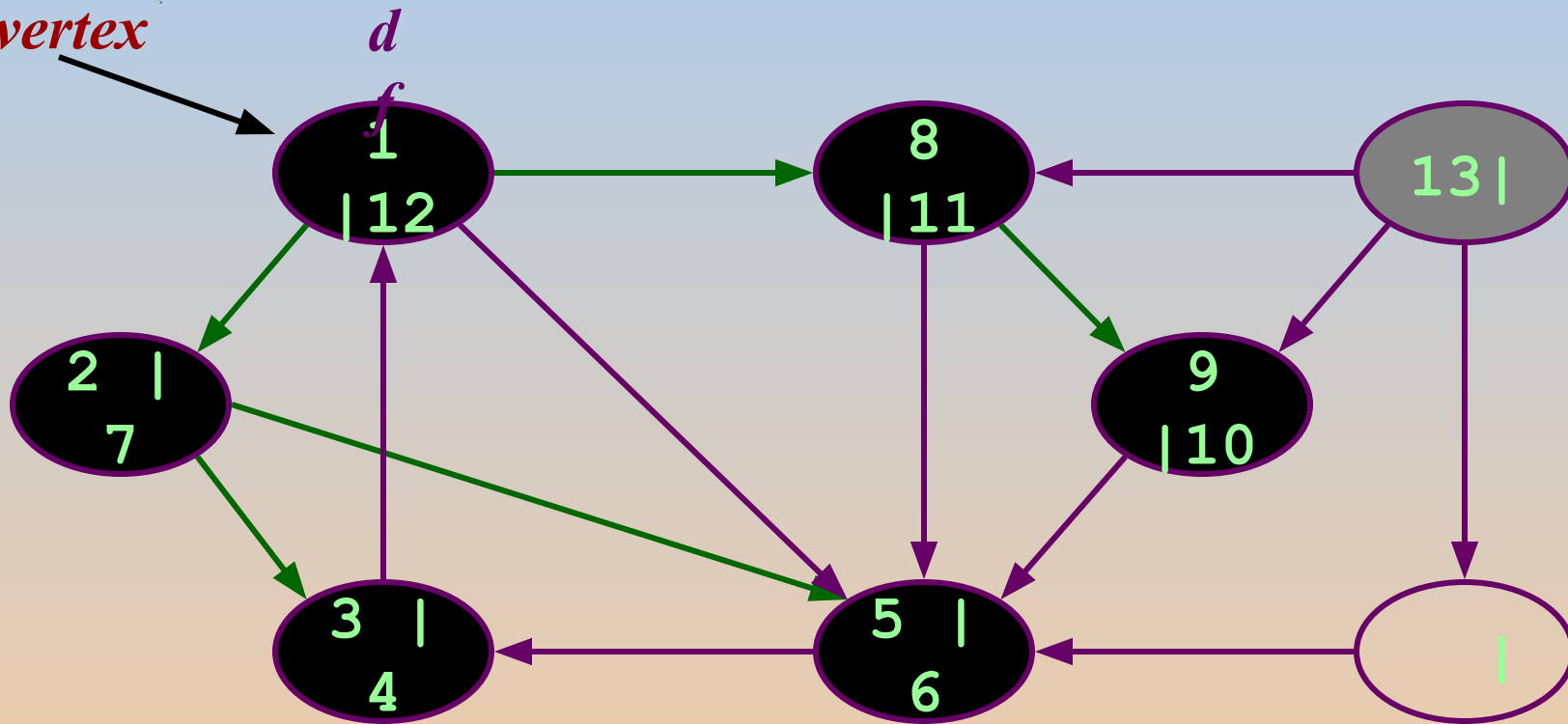
*source
vertex*



DFS Example

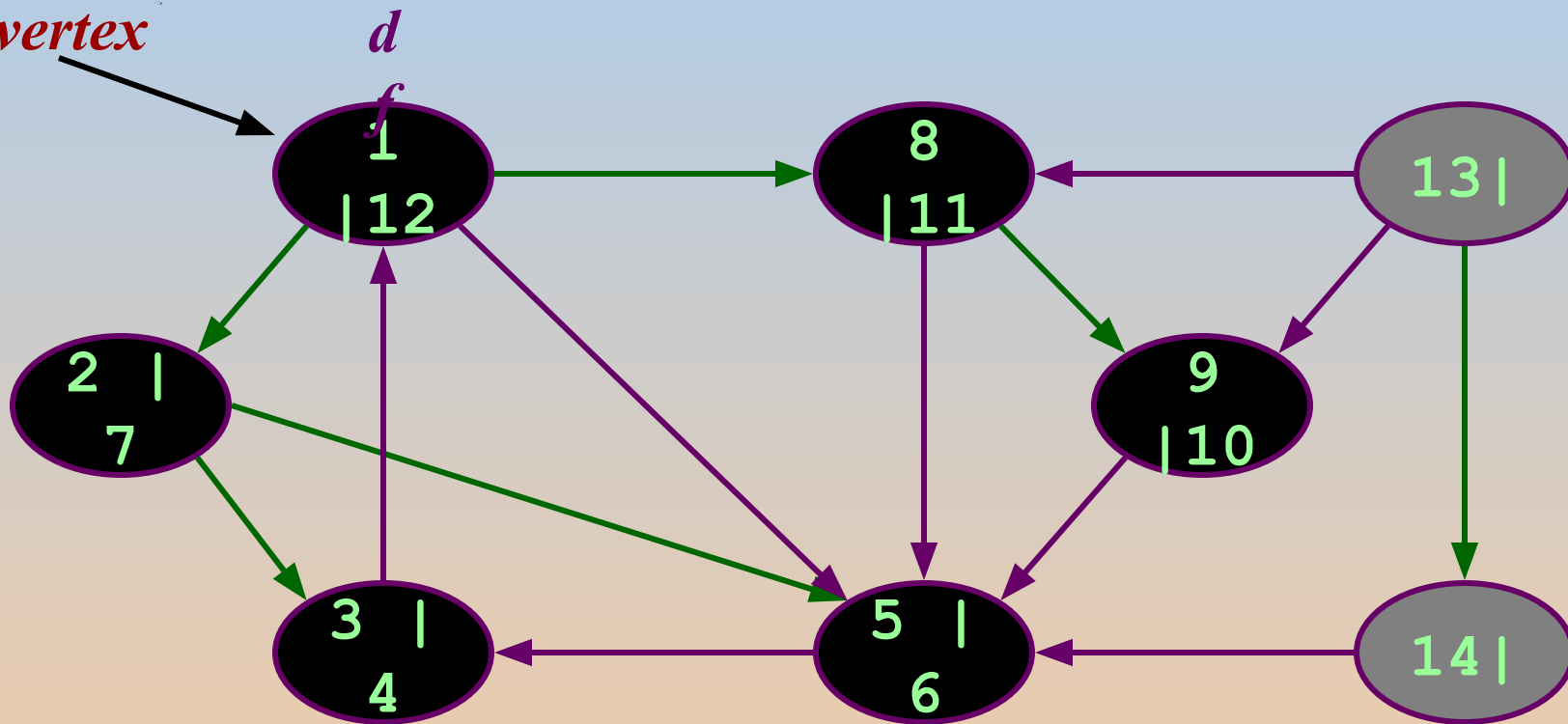


*source
vertex*



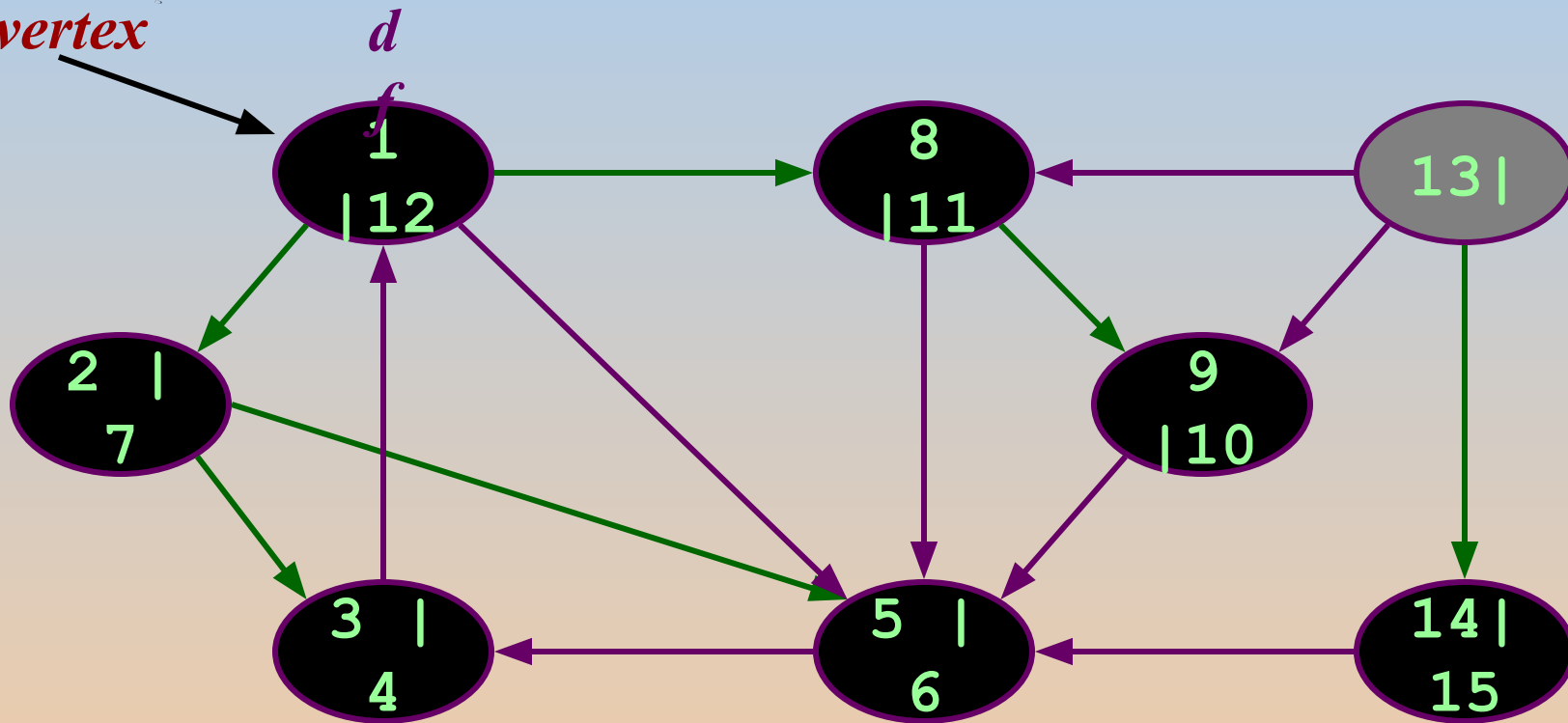
DFS Example

*source
vertex*

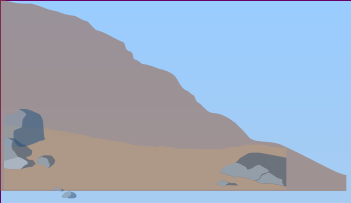


DFS Example

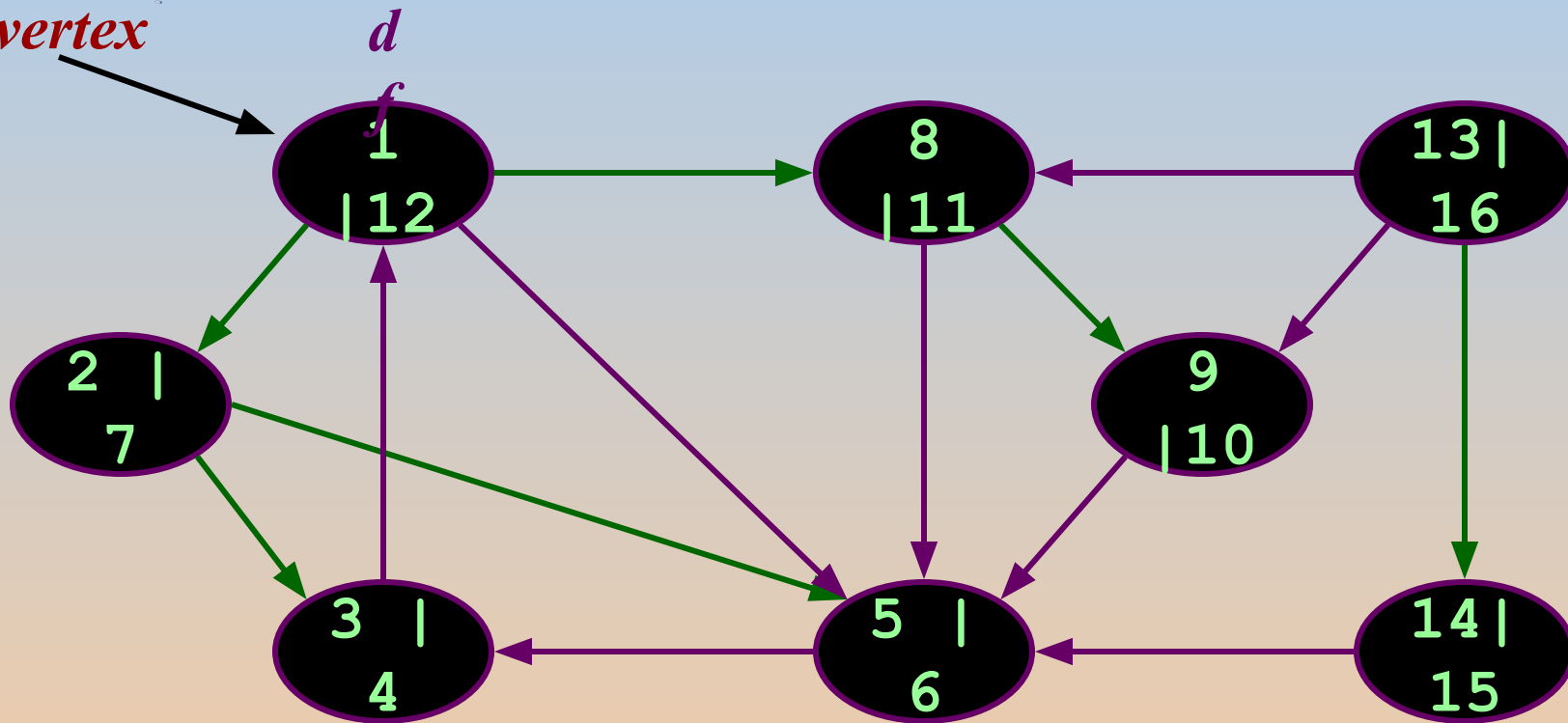
*source
vertex*

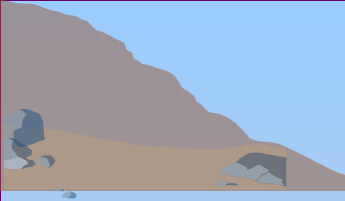


DFS Example



*source
vertex*





DFS: Algorithm

DFS(G)

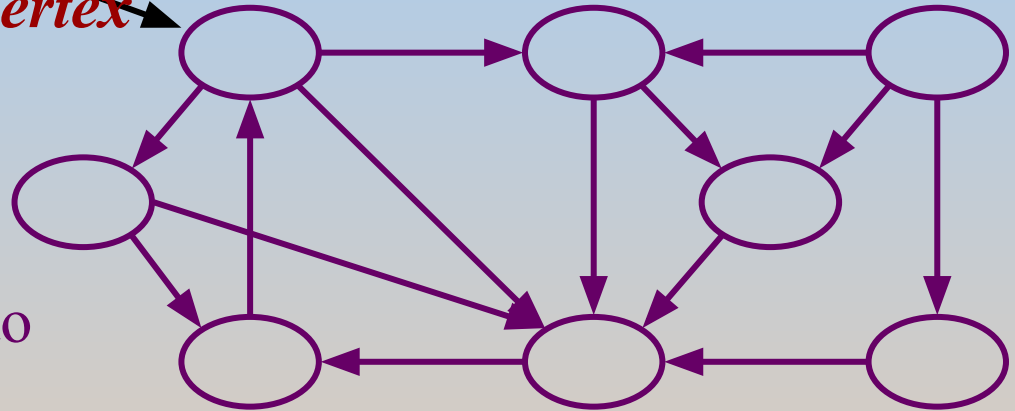
- for each vertex u in V ,
- $\text{color}[u] = \text{white}; \pi[u] = \text{NIL}$
- $\text{time} = 0;$
- for each vertex u in V
 - if ($\text{color}[u] = \text{white}$)
 - DFS-VISIT(u)

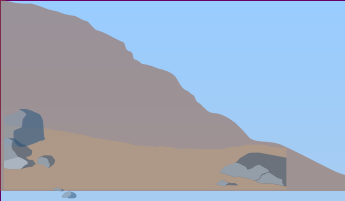
DFS: Algorithm (Cont.)

DFS-VISIT(u)

- $\text{color}[u] = \text{gray};$
- $\text{time} = \text{time} + 1;$
- $d[u] = \text{time};$
- for each v in $\text{Adj}(u)$ do
- if ($\text{color}[v] = \text{white}$)
- $\pi[v] = u;$
- DFS-VISIT(v);
- $\text{color}[u] = \text{black};$
- $\text{time} = \text{time} + 1; f[u] = \text{time};$

*source
vertex* →





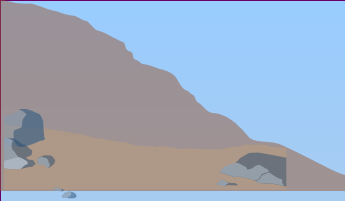
DFS: Complexity Analysis

Initialization complexity is $O(V)$

DFS_VISIT is called exactly once for each vertex

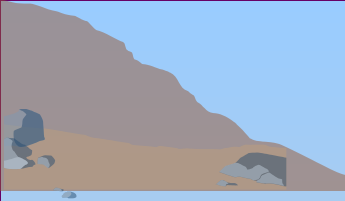
And DFS_VISIT scans all the edges which causes cost of $O(E)$

Overall complexity is $O(V + E)$

A decorative illustration in the top-left corner showing a brown mountain range, a blue sky, and a small body of water with a few rocks.

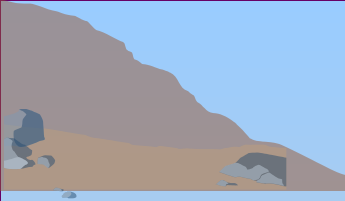
DFS: Application

- Topological Sort
- Strongly Connected Component



Breadth-first Search (BFS)

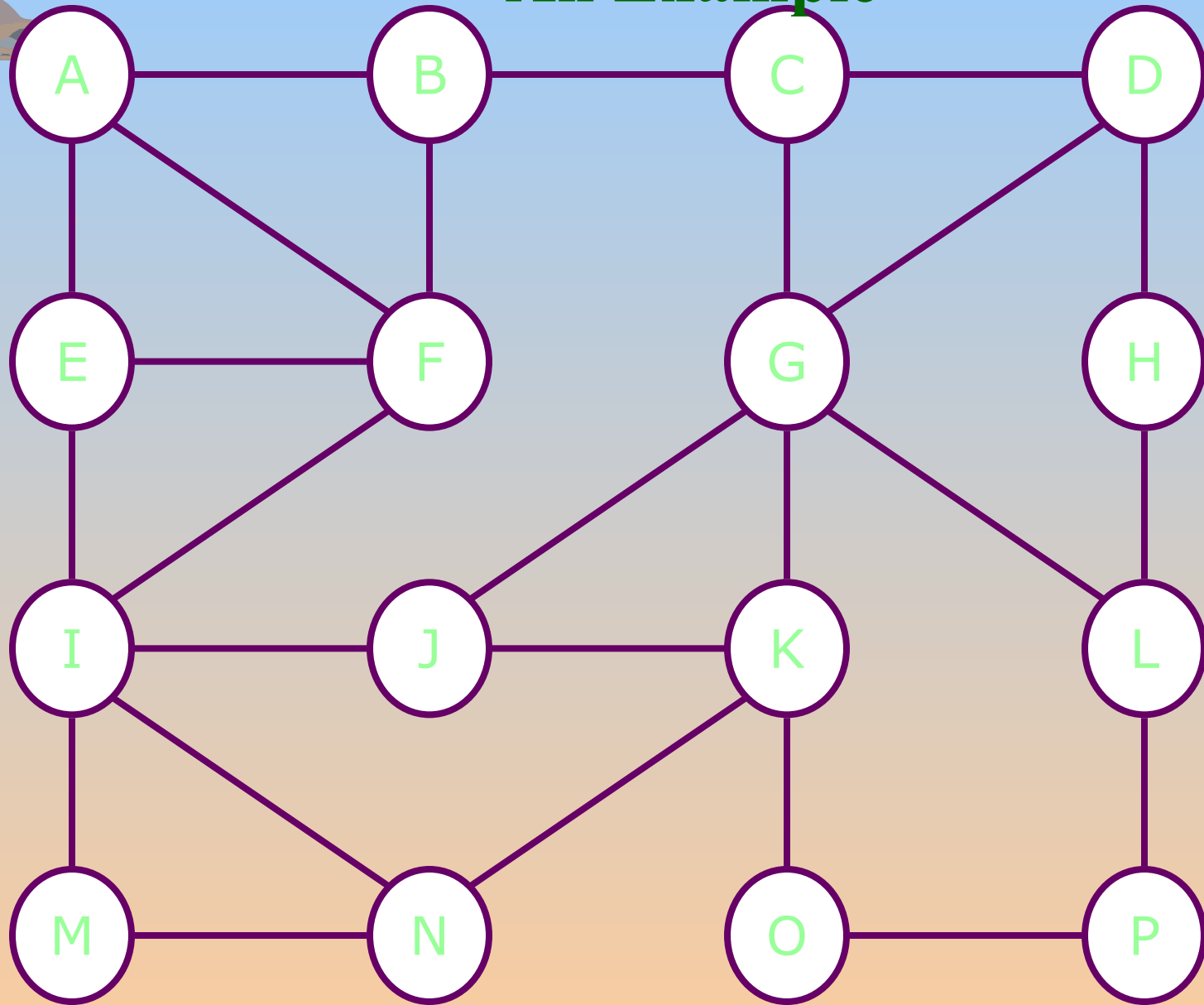
- Search for all vertices that are directly reachable from the root (called level 1 vertices)
- After mark all these vertices, visit all vertices that are directly reachable from any level 1 vertices (called level 2 vertices), and so on.
- In general, level k vertices are directly reachable from a level $k - 1$ vertices

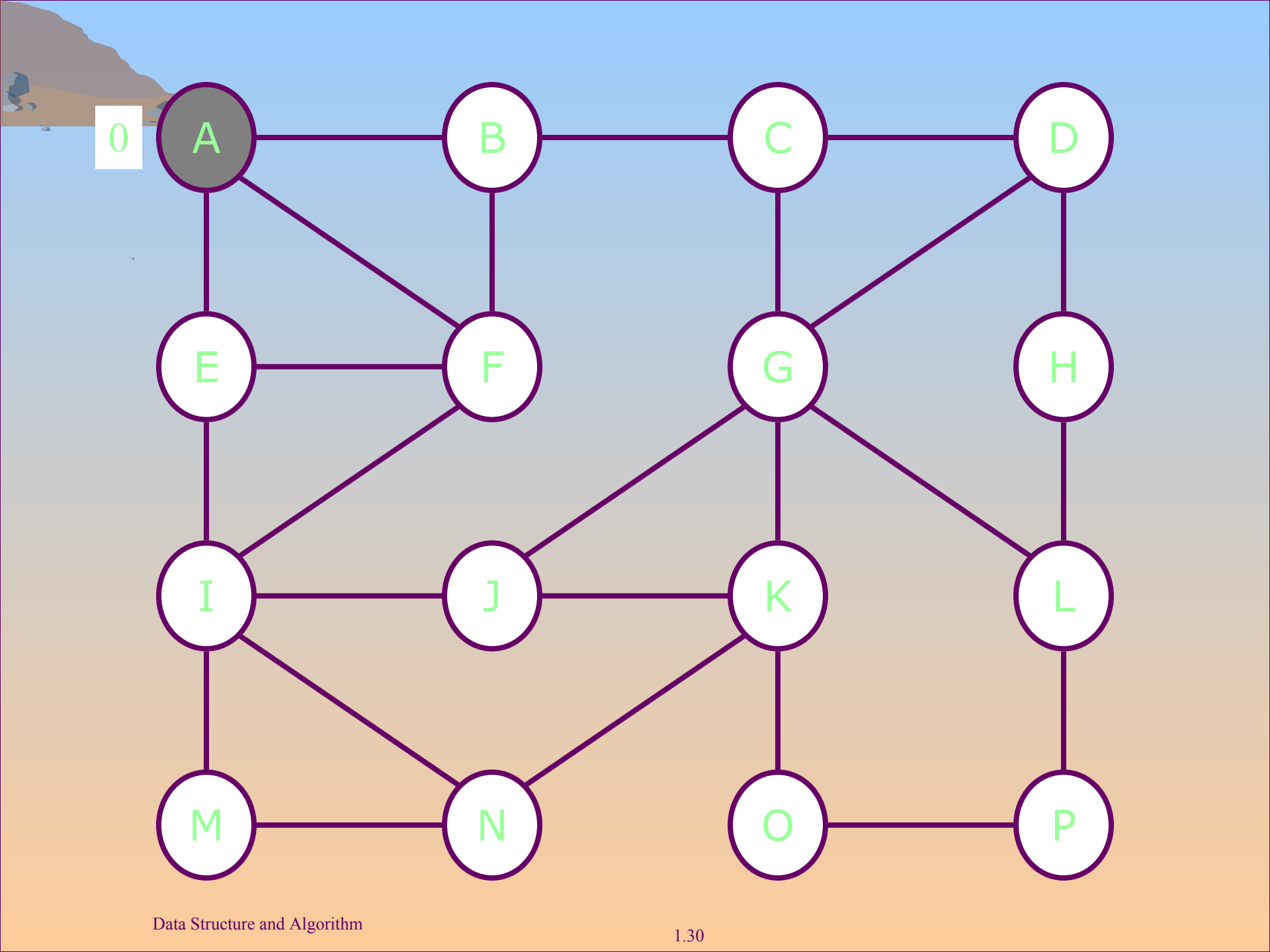


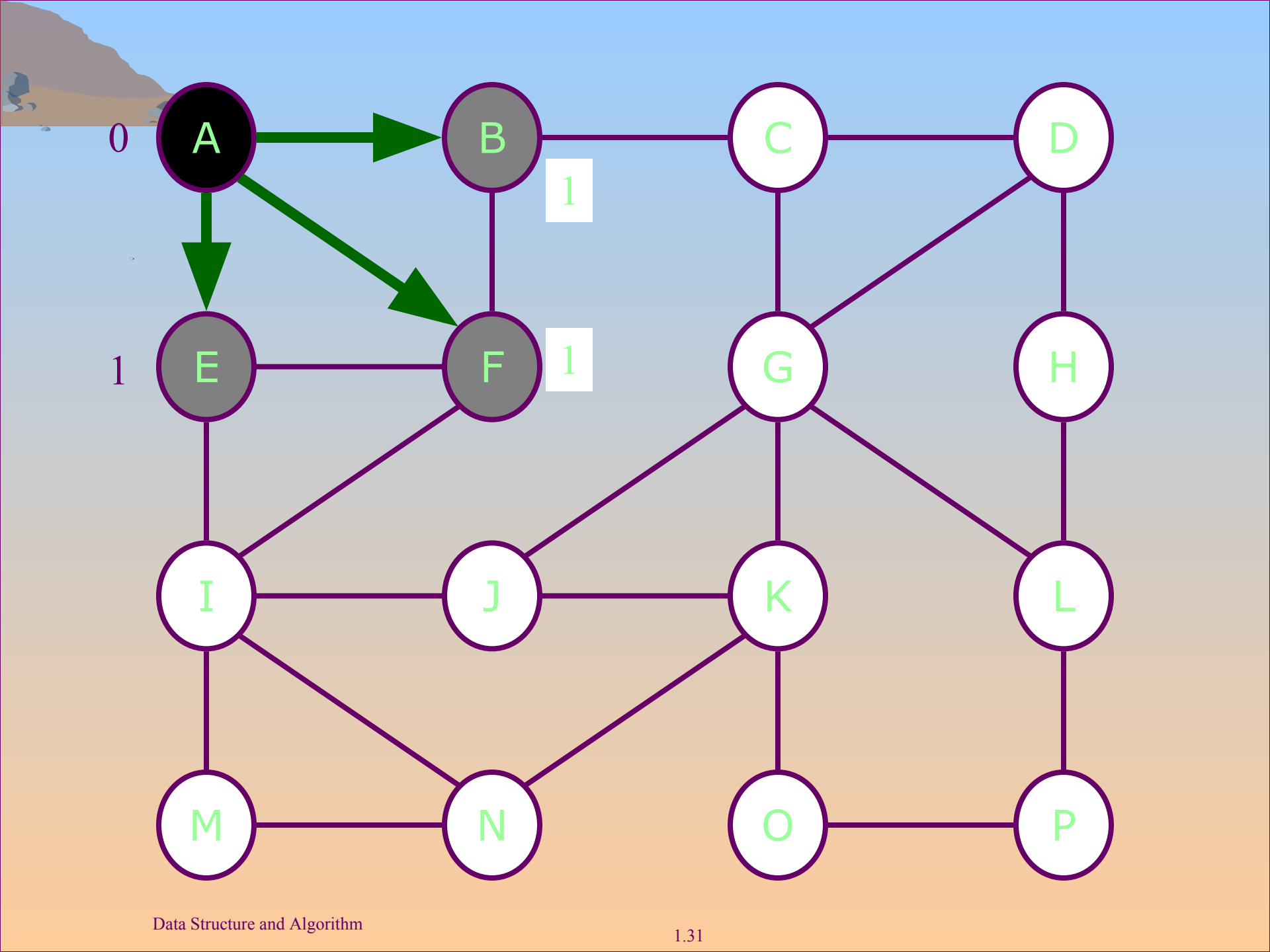
BFS: the Color Scheme

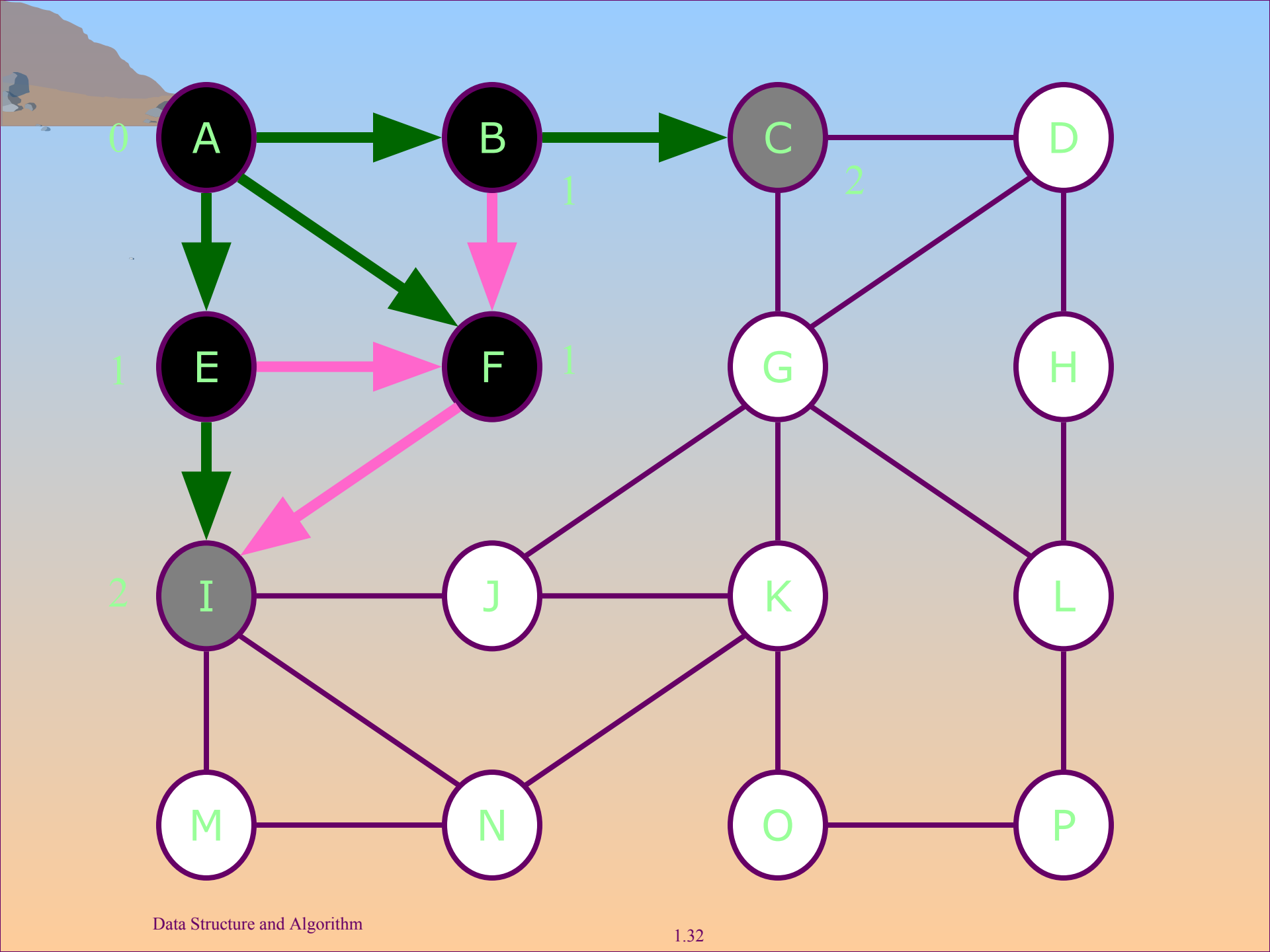
- White vertices have not been discovered
 - All vertices start out white
- Grey vertices are discovered but not fully explored
 - They may be adjacent to white vertices
- Black vertices are discovered and fully explored
 - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

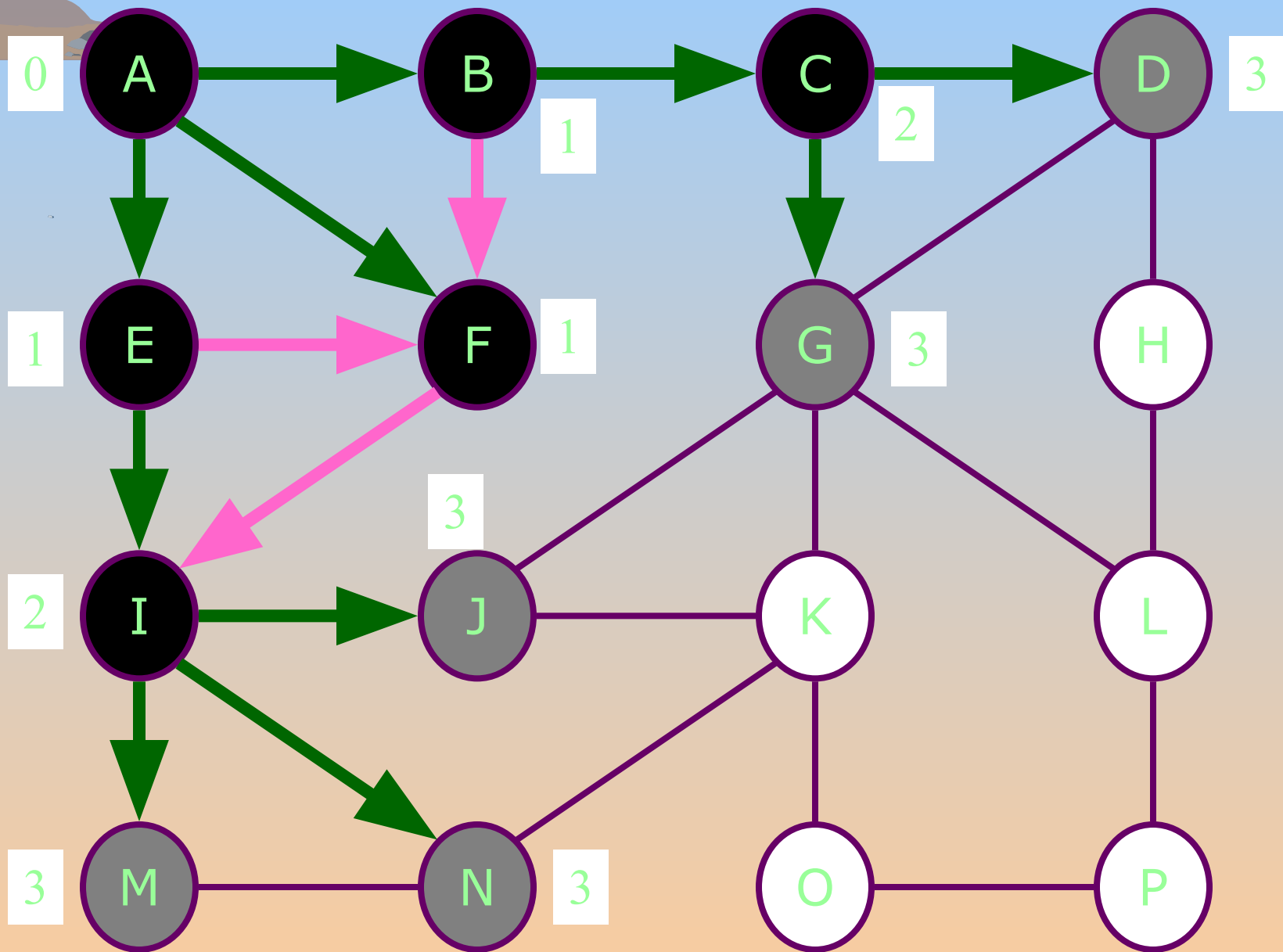
An Example

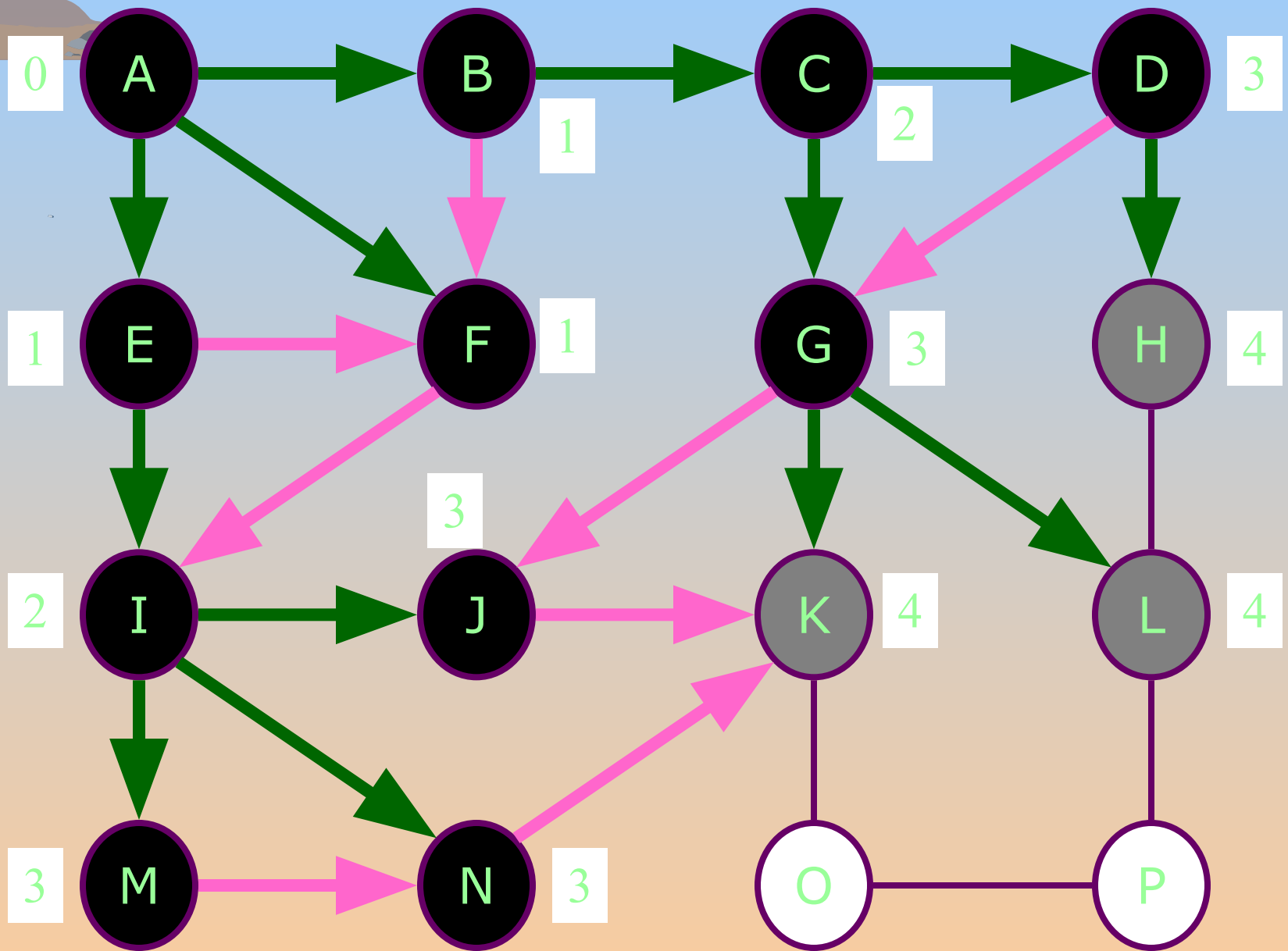


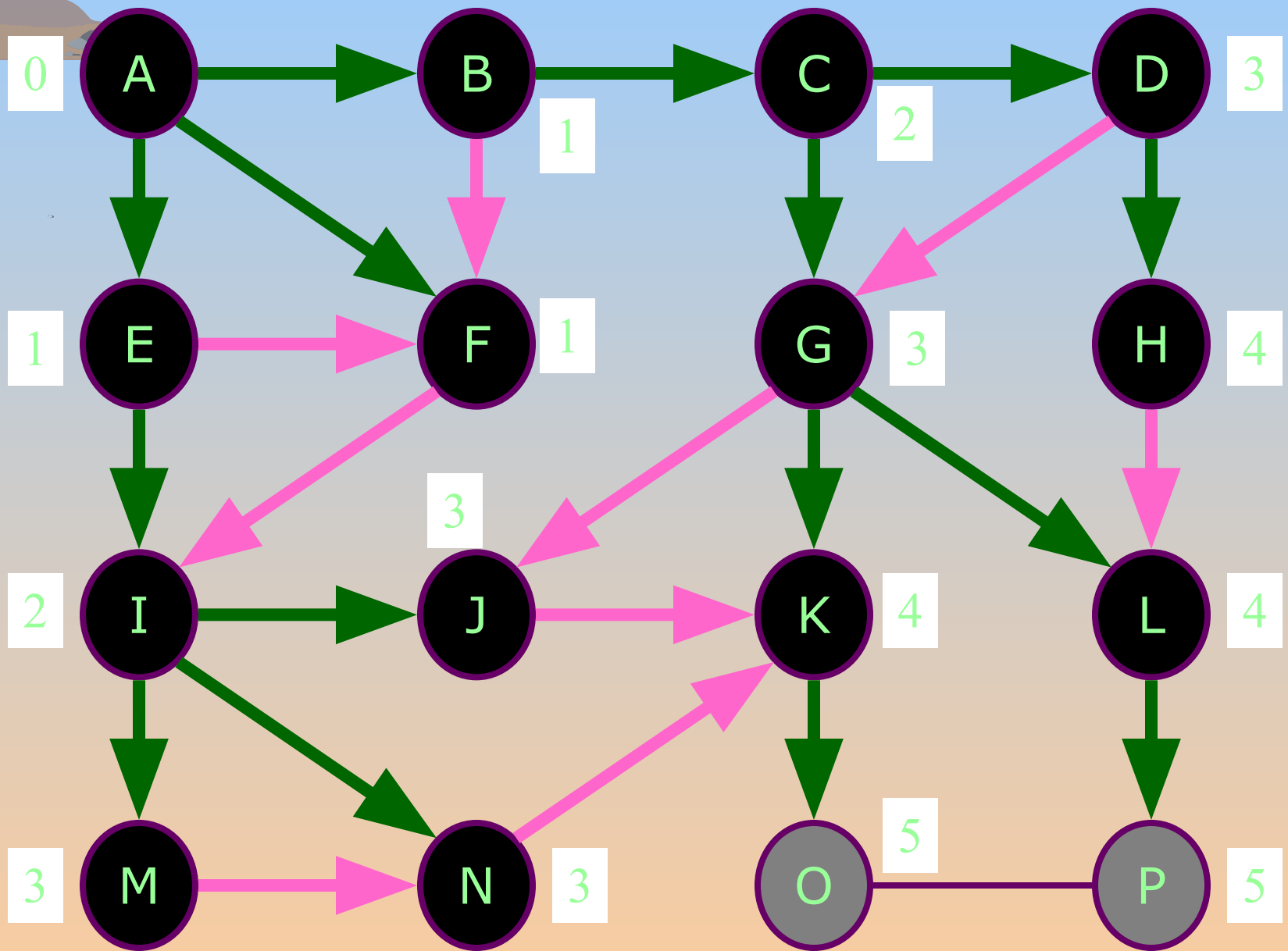


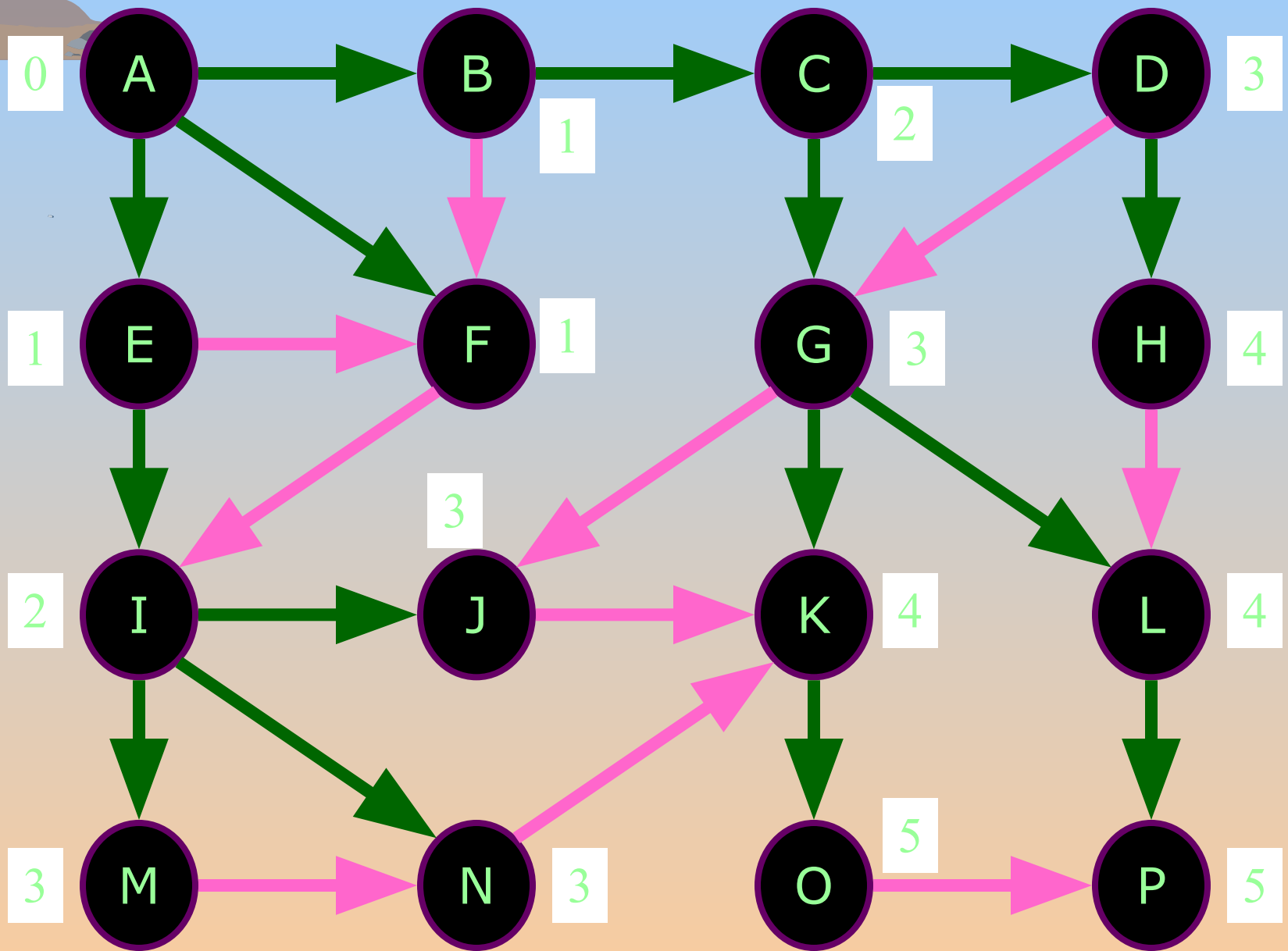


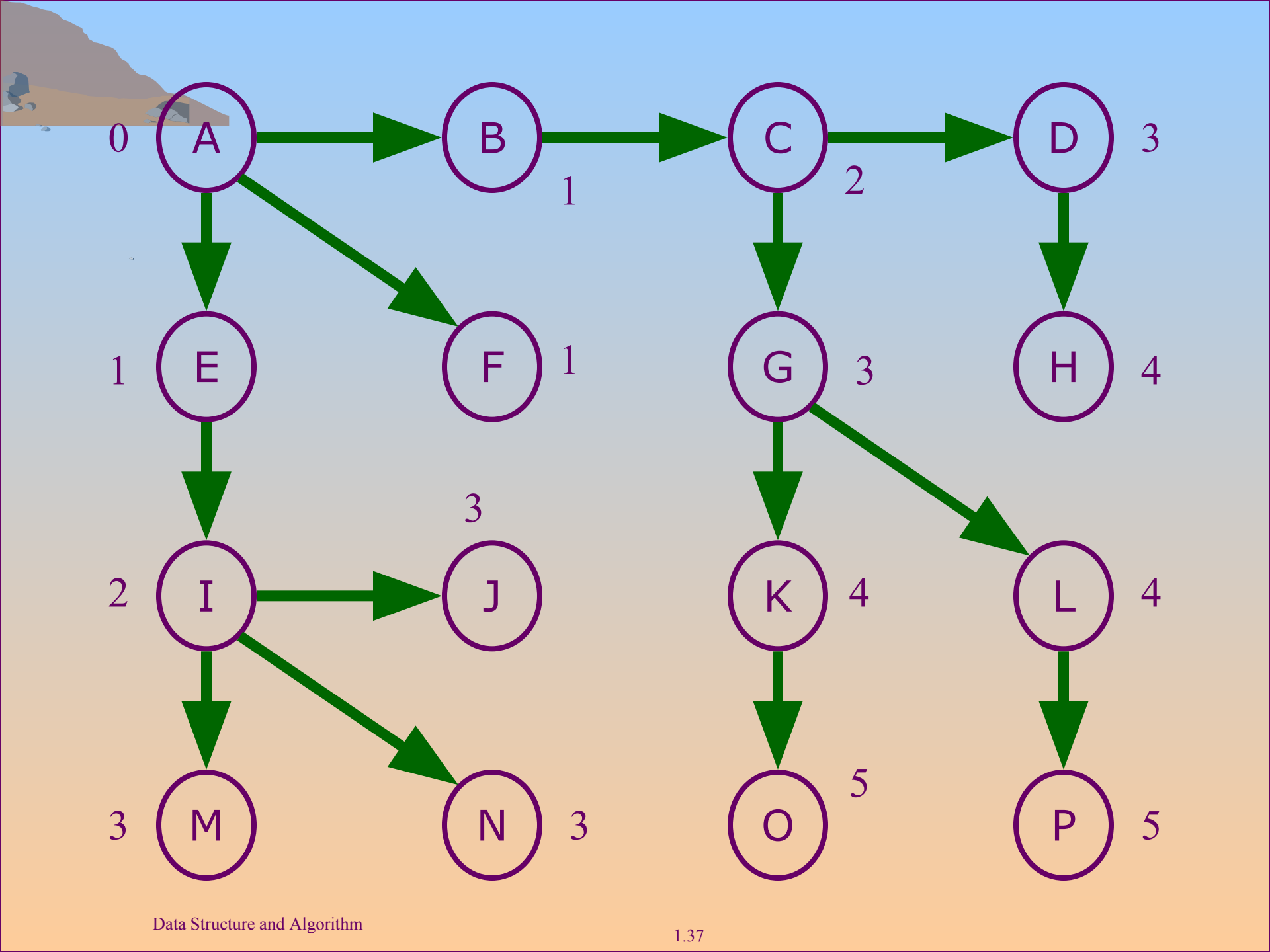










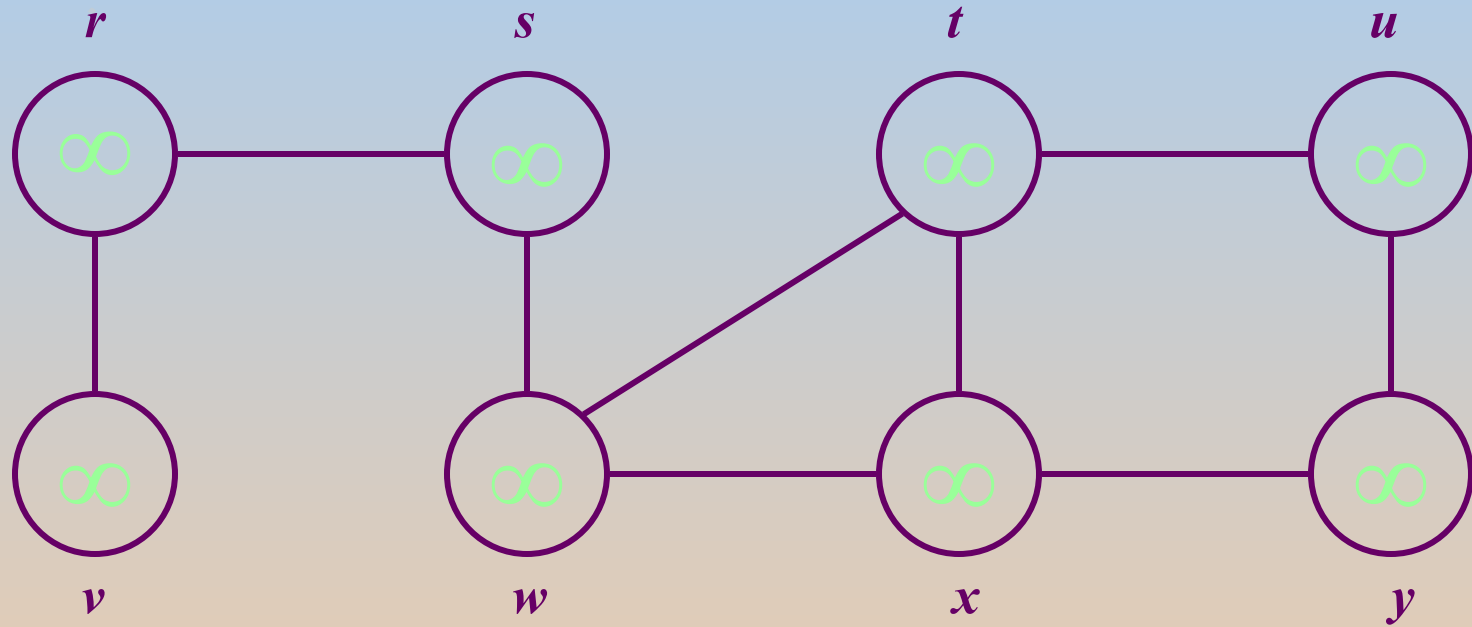


BFS: Algorithm

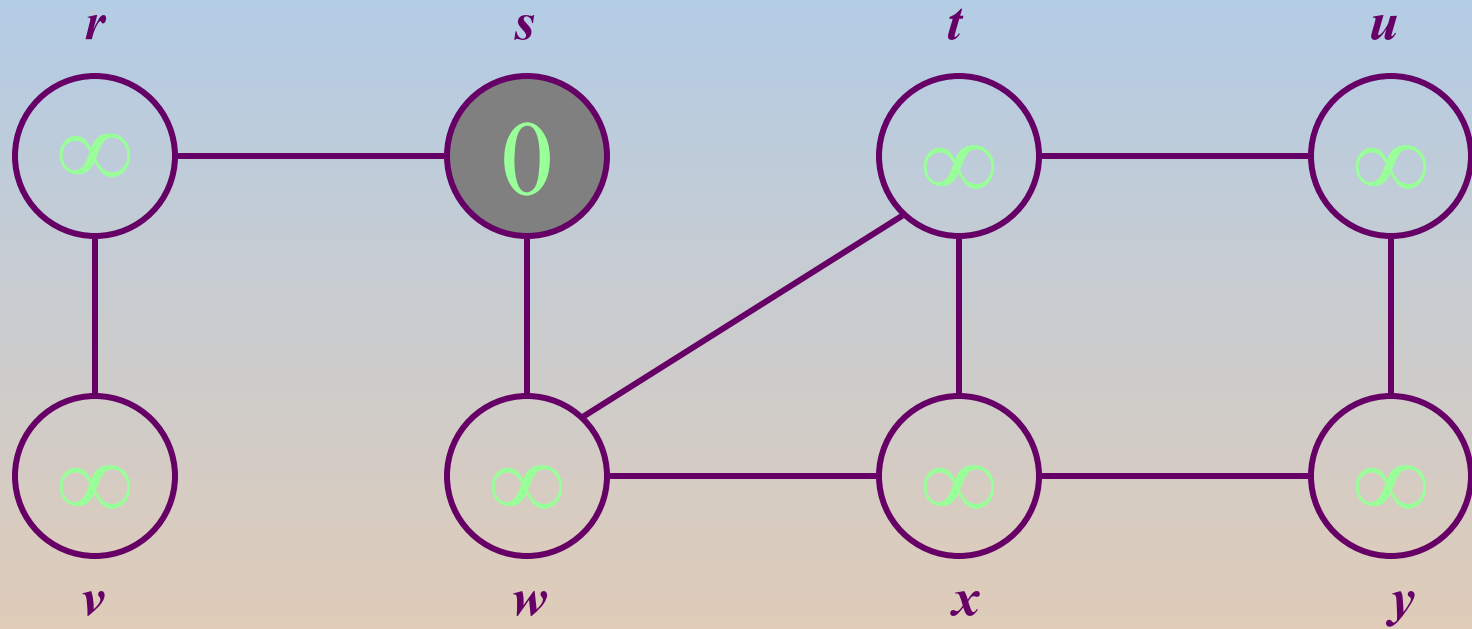
BFS(G, s)

- For each vertex u in $V - \{s\}$,
 - $\text{color}[u] = \text{white};$
 - $d[u] = \text{infinity};$
 - $\pi[u] = \text{NIL}$
- $\text{color}[s] = \text{GRAY}; \quad d[s] = 0; \quad \pi[s] = \text{NIL}; \quad Q = \text{empty queue}$
- $\text{ENQUEUE}(Q, s)$
- while (Q not empty)
- $u = \text{DEQUEUE}(Q)$
- for each $v \in \text{Adj}[u]$
- if $\text{color}[v] = \text{WHITE}$
- then $\text{color}[v] = \text{GREY}$
- $d[v] = d[u] + 1; \pi[v] = u$
- $\text{ENQUEUE}(Q, v);$
- $\text{color}[u] = \text{BLACK};$
-

Example

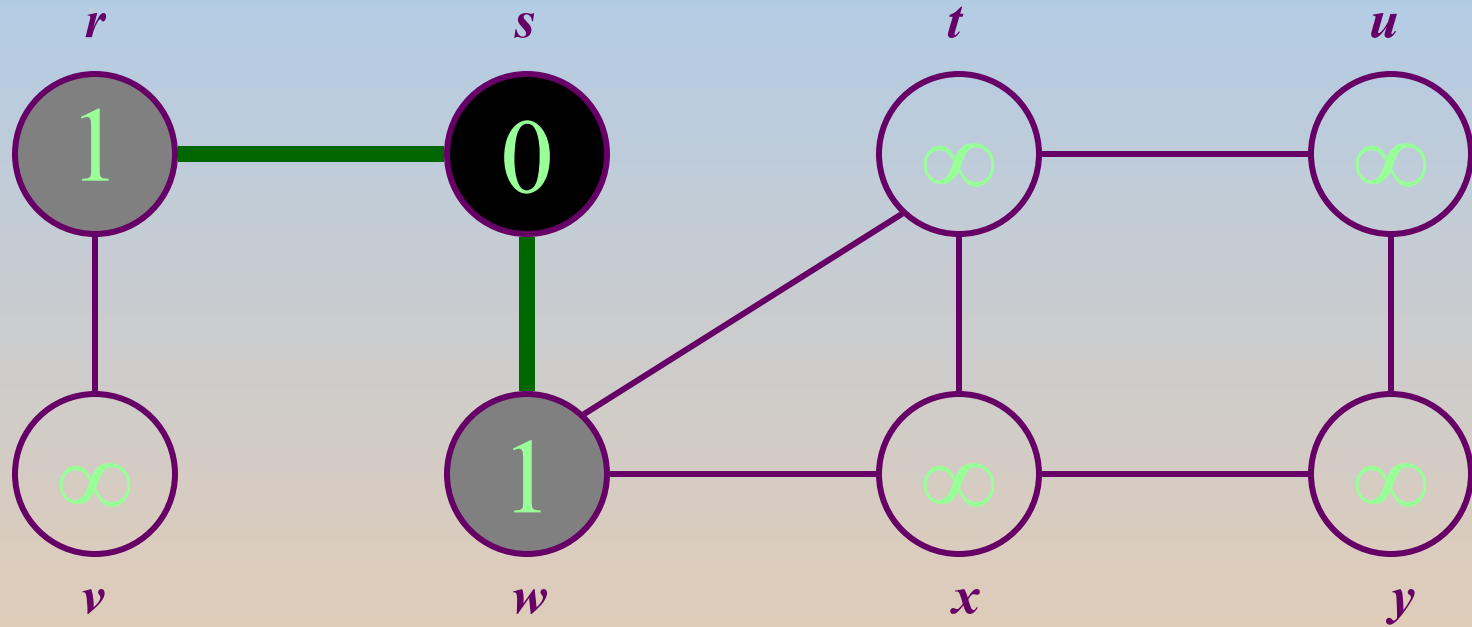


Example



$Q:$ s

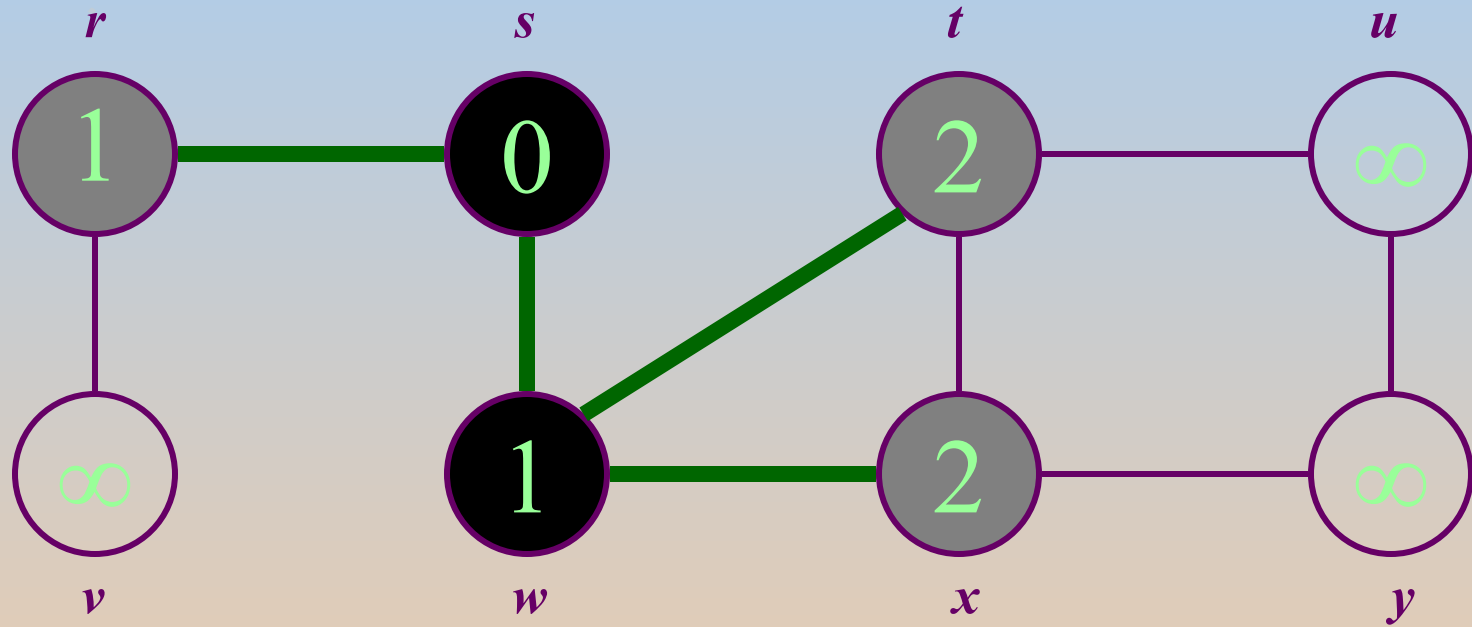
Example



Q :

w	r
-----	-----

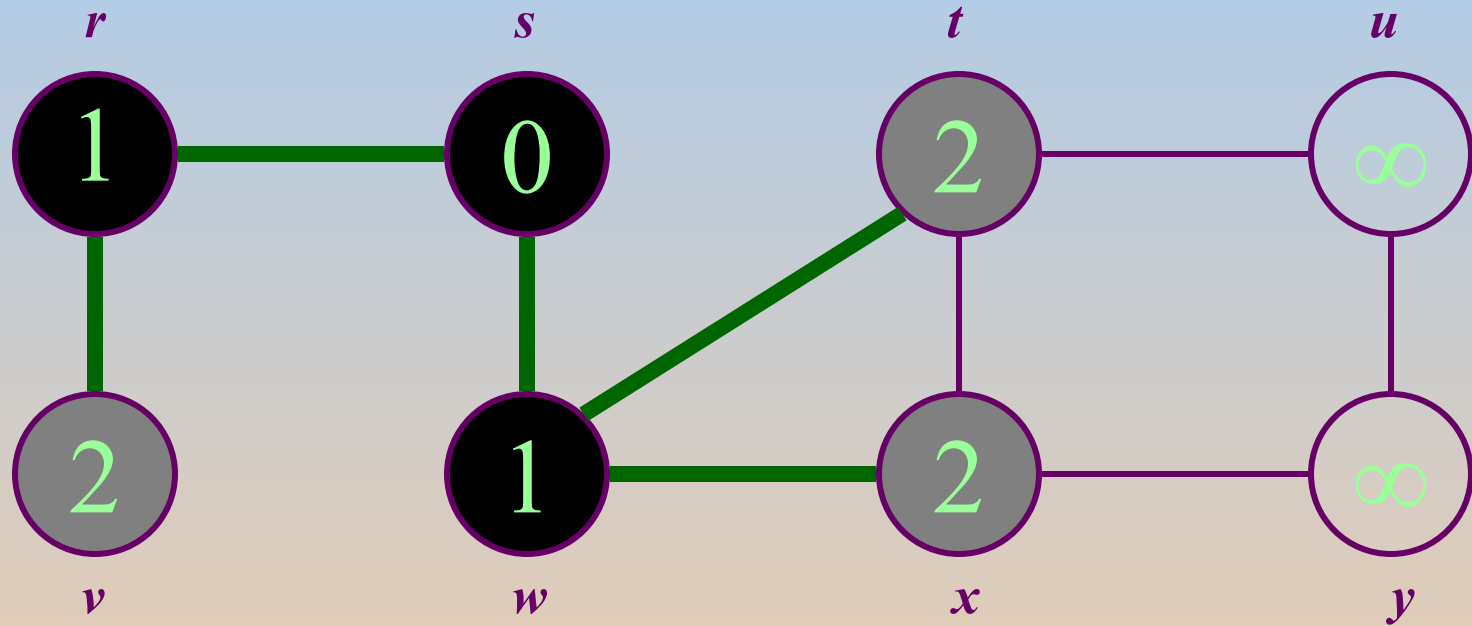
Example



Q :

r	t	x
-----	-----	-----

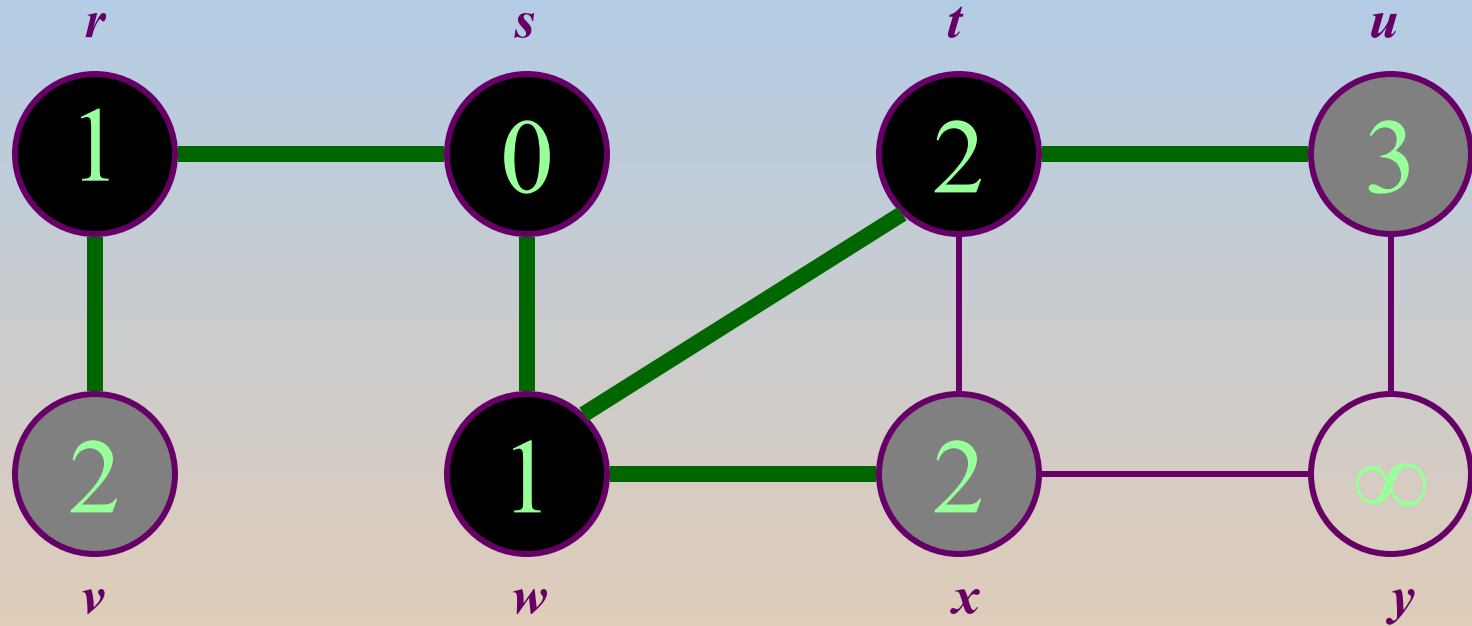
Example



Q :

t	x	v
-----	-----	-----

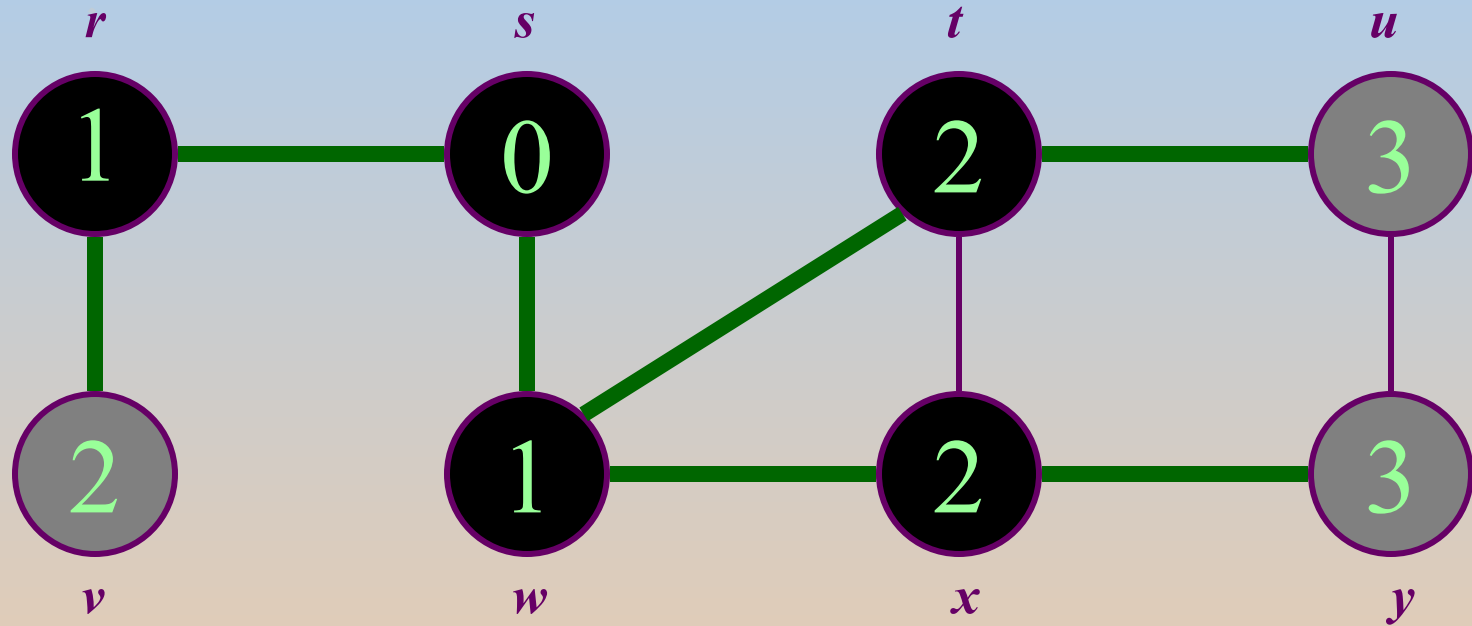
Example



Q :

x	v	u
-----	-----	-----

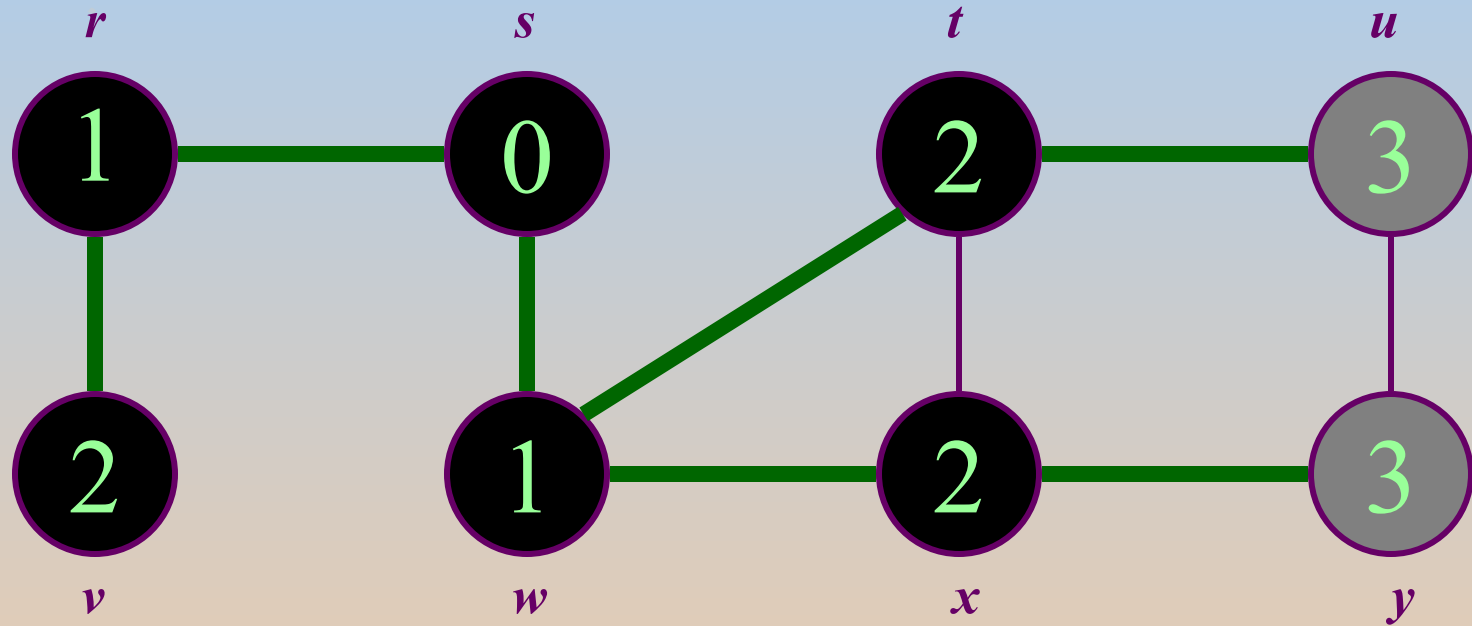
Example



Q :

v	u	y
-----	-----	-----

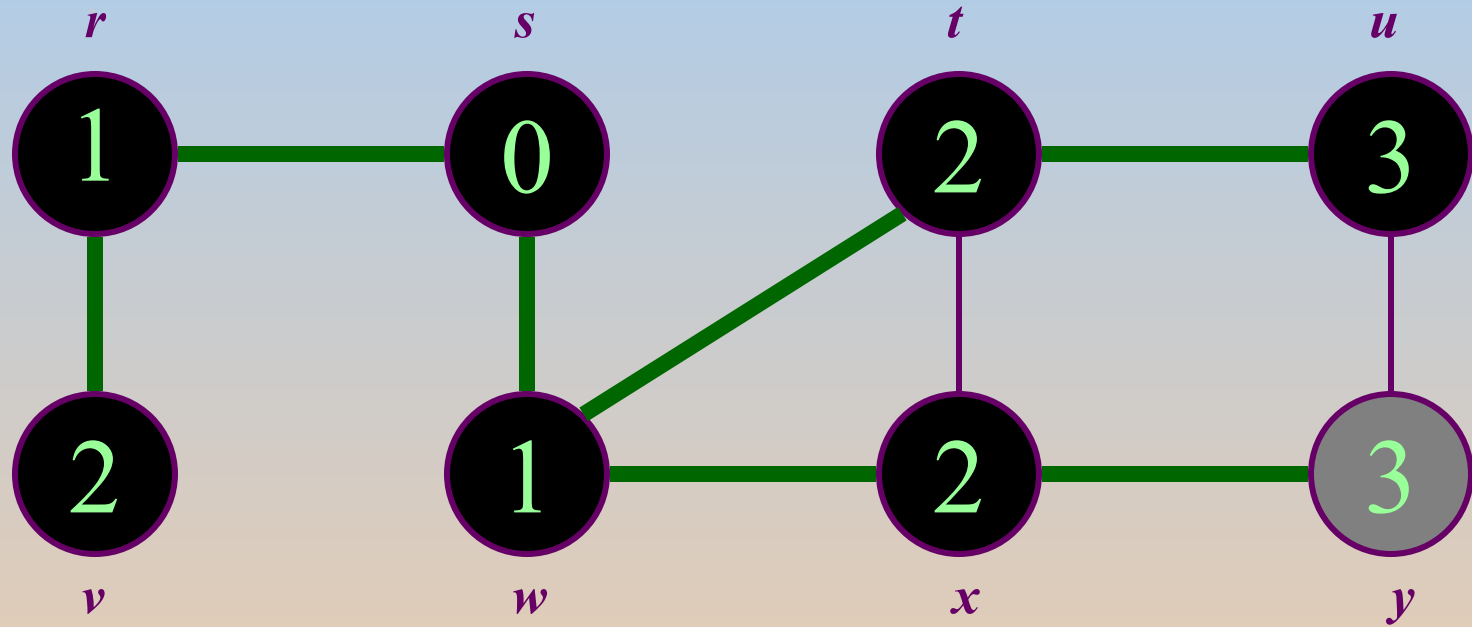
Example



Q :

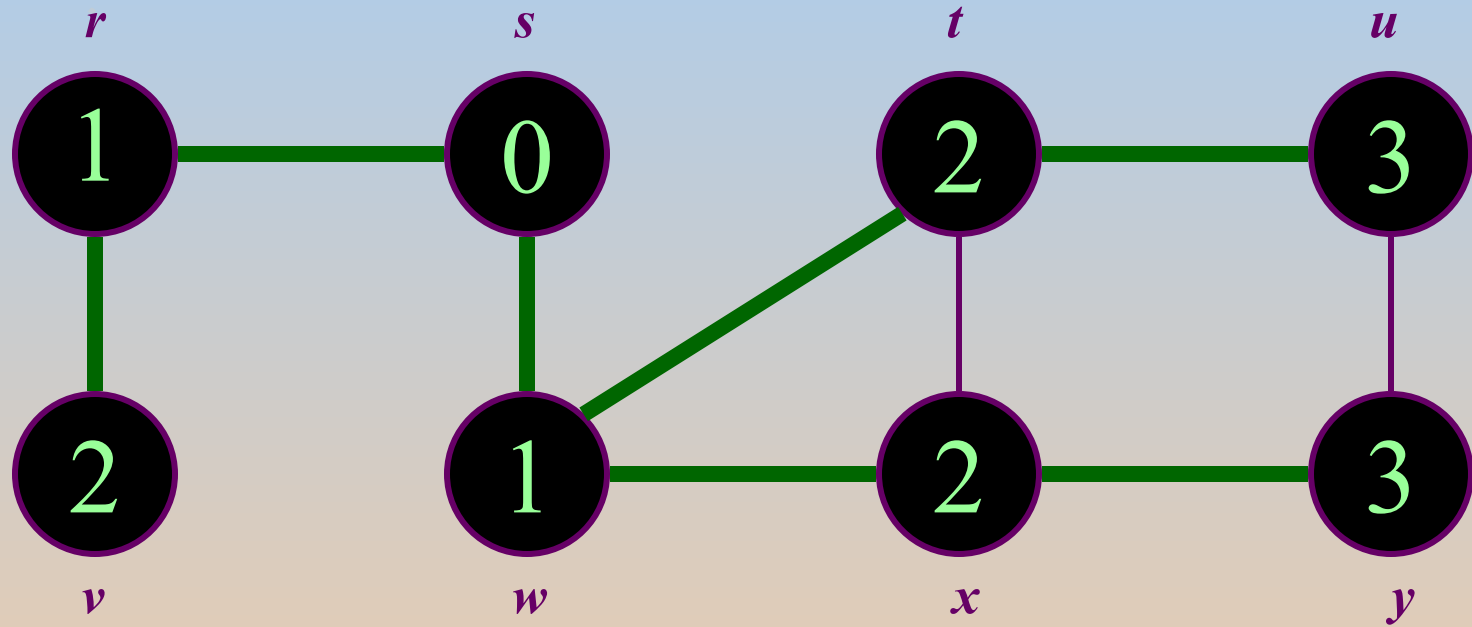
u	y
-----	-----

Example

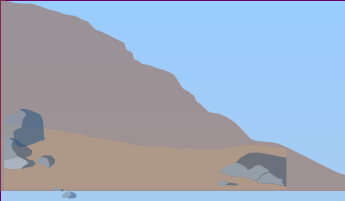


$Q:$ y

Example



$Q: \emptyset$



BFS: Complexity Analysis

- Queuing time is $O(V)$ and scanning all edges requires $O(E)$
- Overhead for initialization is $O(V)$
- So, total running time is $O(V+E)$

BFS: Application

- Shortest path problem